CS-317 Operating Systems

Lecture 26

Deadlocks

Book 2 Chapter 7

From the previous lecture…

- We understood why deadlock prevention is not a good option.
- We realized how deadlock avoidance better utilizes system resources.
- We listed the algorithms for deadlock avoidance.
- We explored the Resource-Allocation-Graph algorithm.
- We explored the banker’s algorithm.

Deadlock Avoidance

Technique and Algorithm

Banker's Algorithm

Is this state safe?

Process	Allocation			Need		
	A	B	C	A	B	C
P0	0	1	0	7	4	3
P1	2	0	0	1	2	2
P2	3	0	2	6	0	0
P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

Available		
A	B	C
3	3	2

- We were given a system with Allocation, Max and Available as shown here.
- We computed the Need matrix.
- We found that the given state is safe.
- The safe sequence is:

$$< P_1, P_3, P_4, P_0, P_2 >$$

In this safe state, P_1 additionally requests 1 instance of A and 2 instances of C.

$Request_1 = (1, 0, 2)$

request vector is created at first
the original need is 3,2,2 taken from max of p1
the available was 3,3,2
req<avilabale, req<need hence, assume granted

Can $Request_1$ be granted?

Process	Allocation			Need		
	A	B	C	A	B	C
P0	0	1	0	7	4	3
P1	3	0	2	0	2	0
P2	3	0	2	6	0	0
P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

$Request_1$		
A	B	C
1	0	2

Available		
A	B	C
2	3	0

- Run the resource-request algorithm.
- $Request_1 \leq Need_1$
- $Request_1 \leq Available$
- $Available = Available - Request_1$
- $Allocation_1 = Allocation_1 + Request_1$
- $Need_1 = Need_1 - Request_1$

avilable = 3,3,2 - 1,0,2 (av - req)
allocation = 2,0,0(all) + 1,0,2(req)
p1 need = 1,2,2(need) - 1,0,2

We have assumed that $Request_1$ is granted and the state has been changed accordingly.

Find out if the resulting state is safe.

Is this state safe?

Process		Allocation			Need		
	Finish	A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	F	3	0	2	0	2	0
P2	F	3	0	2	6	0	0
P3	F	2	1	1	0	1	1
P4	F	0	0	2	4	3	1

Available		
A	B	C
2	3	0

Work		
A	B	C
2	3	0

- Run the safety algorithm.
- Set all finish to False.
- Set Work = Available.

Is this state safe?

Process		Allocation			Need		
	Finish	A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	T	3	0	2	0	2	0
P2	F	3	0	2	6	0	0
P3	F	2	1	1	0	1	1
P4	F	0	0	2	4	3	1

Work		
A	B	C
5	3	2

- Find an index i such that both Finish[i] == False and $Need_i \leq Work$.
- $Need_1 \leq Work$!
- $Work = Work + Allocation_1$
- Finish[1] = True

Is this state safe?

Process		Allocation			Need		
	Finish	A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	T	3	0	2	0	2	0
P2	F	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	F	0	0	2	4	3	1

Work		
A	B	C
7	4	3

- Find an index i such that both Finish[i] == False and $Need_i \leq Work$.
- $Need_3 \leq Work$!
- $Work = Work + Allocation_3$
- Finish[3] = True

Is this state safe?

Process		Allocation			Need		
	Finish	A	B	C	A	B	C
P0	F	0	1	0	7	4	3
P1	T	3	0	2	0	2	0
P2	F	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	T	0	0	2	4	3	1

Work		
A	B	C
7	4	5

- Find an index i such that both Finish[i] == False and $Need_i \leq Work$.
- $Need_4 \leq Work$!
- $Work = Work + Allocation_4$
- Finish[4] = True

Is this state safe?

Process		Allocation			Need		
	Finish	A	B	C	A	B	C
P0	T	0	1	0	7	4	3
P1	T	3	0	2	0	2	0
P2	F	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	T	0	0	2	4	3	1

Work		
A	B	C
7	5	5

- Find an index i such that both Finish[i] == False and $Need_i \leq Work$.
- $Need_0 \leq Work$!
- $Work = Work + Allocation_0$
- Finish[0] = True

Is this state safe?

Process		Allocation			Need		
	Finish	A	B	C	A	B	C
P0	T	0	1	0	7	4	3
P1	T	3	0	2	0	2	0
P2	T	3	0	2	6	0	0
P3	T	2	1	1	0	1	1
P4	T	0	0	2	4	3	1

Work		
A	B	C
10	5	7

- Find an index i such that both Finish[i] == False and $Need_i \leq Work$.
- $Need_2 \leq Work$!
- $Work = Work + Allocation_2$
- Finish[2] = True

Is this state safe?

Process	Allocation			Need		
	A	B	C	A	B	C
P0	0	1	0	7	4	3
P1	3	0	2	0	2	0
P2	3	0	2	6	0	0
P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

Available		
A	B	C
2	3	0

Work		
A	B	C
10	5	7

- Finish[i] == True for all i
- So the state is safe.
- The safe sequence is:

  $< P_1, P_3, P_4, P_0, P_2 >$

- $Request_1$ can be granted.
- This is the new state of the system after grant of $Request_1$.

In this safe state, P_4 additionally requests 3 instances of A and 3 instances of B.

$Request_4 = (3, 3, 0)$

Can $Request_4$ be granted?

Process	Allocation			Need		
	A	B	C	A	B	C
P0	0	1	0	7	4	3
P1	3	0	2	0	2	0
P2	3	0	2	6	0	0
P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

Available		
A	B	C
2	3	0

Available		
A	B	C
2	3	0

havent subtracted and added anything

- Run the resource-request algorithm.
- $Request_4 \leq Need_4$
- $Request_4 > Available$! should have been less
- $Request_4$ cannot be granted!
- The system state is unchanged.

In this safe state, P_0 additionally requests 2 instances of B.

$Request_0 = (0, 2, 0)$

Can $Request_0$ be granted?

Process	Allocation			Need		
	A	B	C	A	B	C
P0	0	1	0	7	4	3
P1	3	0	2	0	2	0
P2	3	0	2	6	0	0
P3	2	1	1	0	1	1
P4	0	0	2	4	3	1

$Request_0$		
A	B	C
0	2	0

Available		
A	B	C
2	3	0

- Run the resource-request algorithm to see if $Request_0$ can be granted.
- If it can be granted, run the safety algorithm to see if the resulting state is safe.
- If the state is safe, furnish a safe sequence and show the new state.

no safe sequnce xsit

Homework

Process	Allocation				Max			
	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2
P1	1	0	0	0	1	7	5	0
P2	1	3	5	4	2	3	5	6
P3	0	6	3	2	0	6	5	2
P4	0	0	1	4	0	6	5	6

Available			
A	B	C	D
1	5	2	0

- Consider a system with 5 processes P_0 through P_4 and 4 resource types A, B, C and D.
- Is the state safe?
- P_1 requests 4 and 2 instances of B and C, respectively. Can the request be granted?

Deadlocks

Book 3 Chapter 10

Deadlock Detection and Recovery

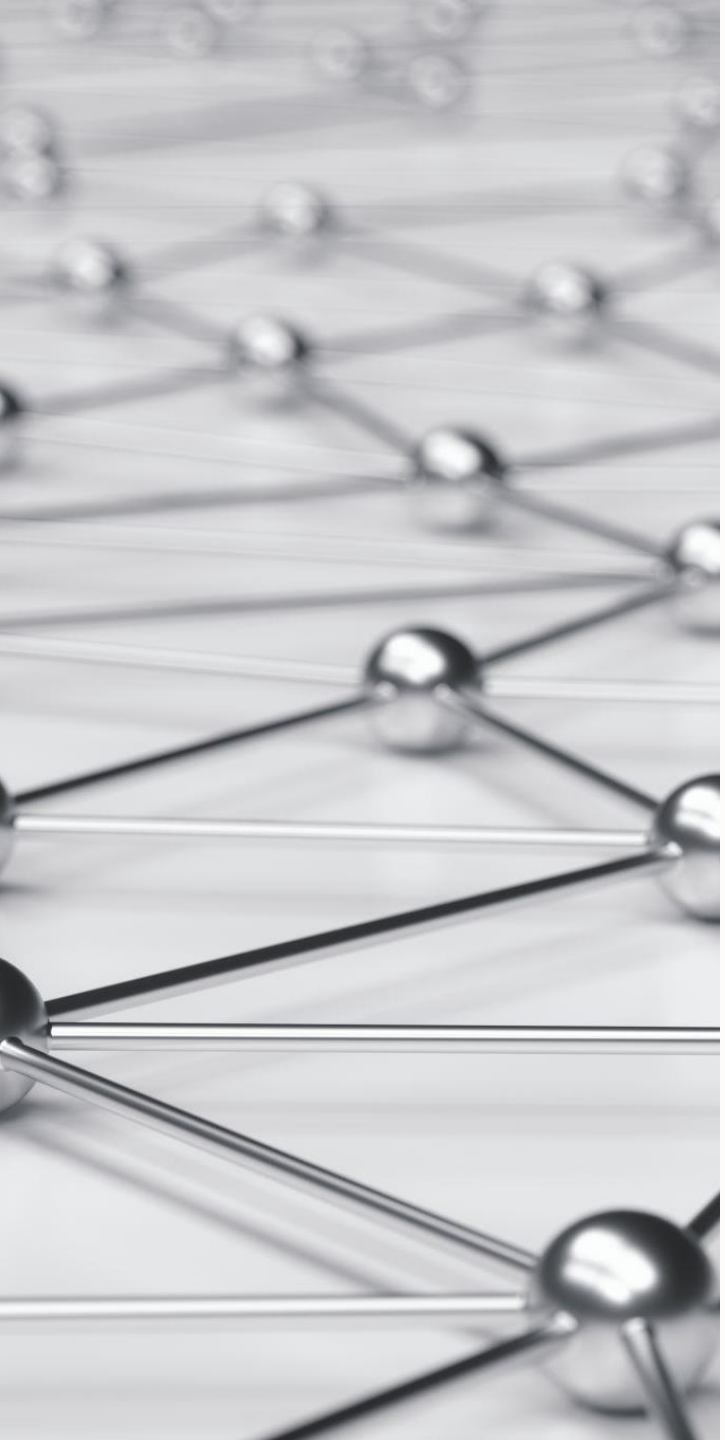
Technique and Algorithm

Why shift from avoidance?

- Whereas avoidance algorithms avoid unsafe states even if the system might recover from them, detection and recovery algorithms ignore the distinction between safe and unsafe states and the resource managers can allocate units whenever they are available.
- If processes seem to be blocked on resources for an inordinately long time, the detection algorithm is executed to determine whether the current state is a deadlock.
- Detection algorithms determine if there is any sequence of transitions in which every process can become unblocked.

The cheapest solution to deadlock

- Prevention implements policies at design time, which limits resources from being allocated even when they are free.
- Avoidance grants or denies requests at runtime, which still leads to the situations where free resources are not allocated. However, the problem and its consequences are less severe than in prevention.
- Avoidance requires the function of the safety and resource-request algorithms in the background throughout system execution, which obviously uses processing time.
- Detection and recovery does not stop any free resource from being allocated, and no background algorithm is run. The algorithm only starts to work when the system seems inactive for a long time.



A revision of resources

- Resources have been defined as "anything a process needs to proceed".
- Serially reusable resources are those resources that a process requests the operating system to allocate exclusively, and the process will later release the resource for another process to reuse.
- Processes also use consumable resources, which cease to exist, once used.
- Instances of the two resource classes are treated differently by the resource manager and processes and have different theoretical properties with respect to deadlock analysis.

Reusable resource graph models

- A reusable resource graph is a directed graph such that the following is true:
- The n + m nodes represent n processes and m resources.
- An edge from P_i to R_j is a request edge, which represents a request for 1 unit of R_j by P_i.
- An edge from R_j to P_i is an assignment edge, which represents the allocation of 1 unit of R_j to P_i.
- For each R_j, there is a count of the number of units of the type, c_j, that is graphically represented by small tokens inside R_j.

The circular wait

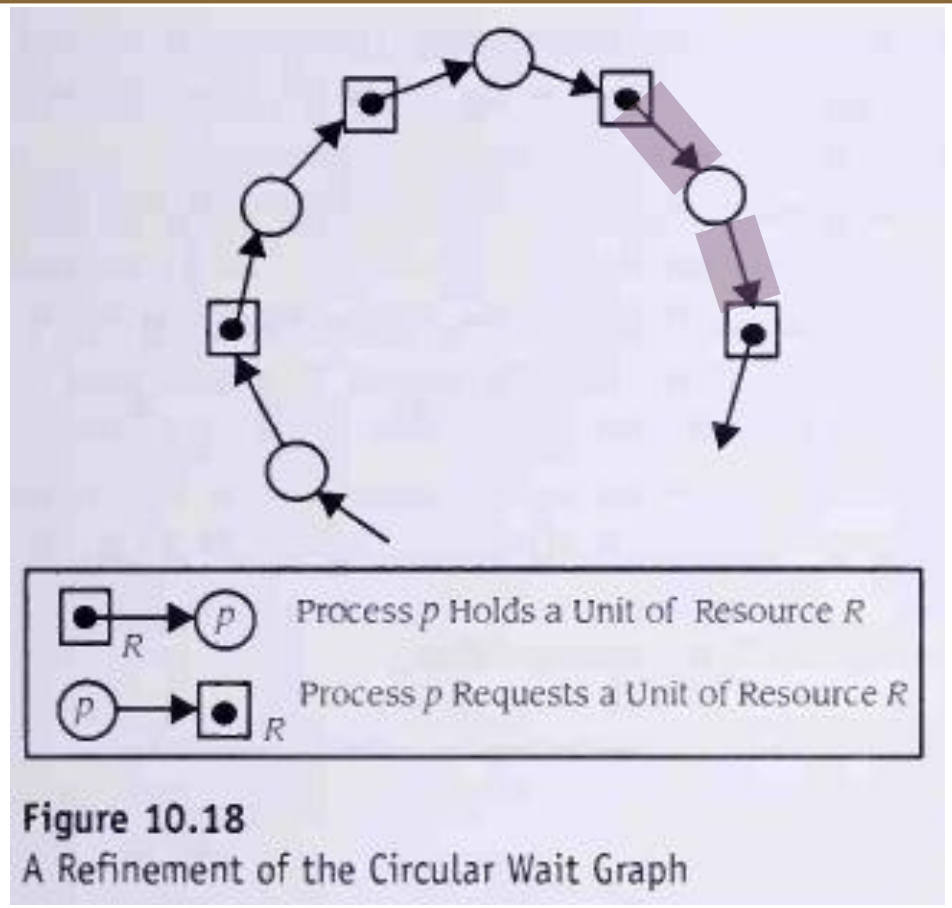
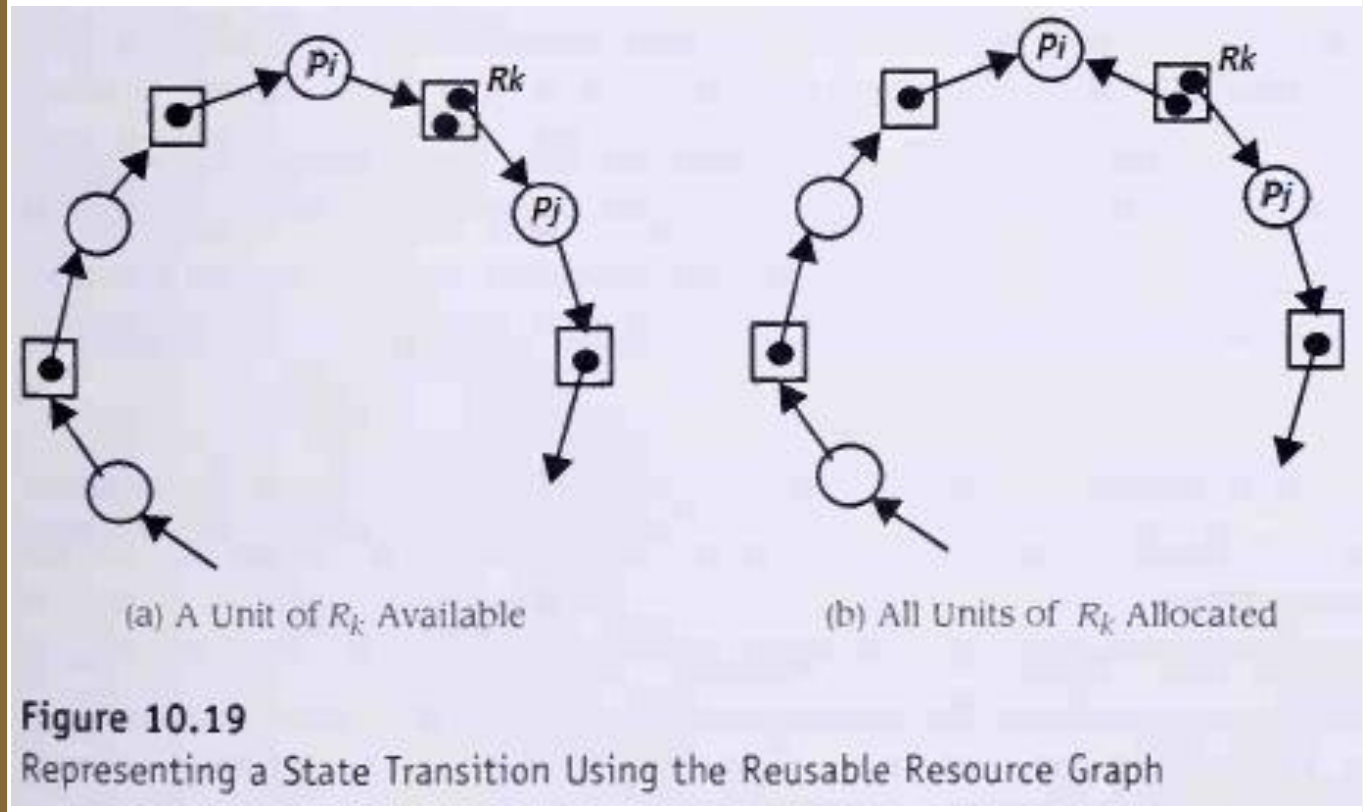


Figure 10.18
A Refinement of the Circular Wait Graph

- This is a reusable resource graph, depicting the circular wait.
- There is only one instance of each resource type, which is held by a process, indicated by an assignment edge.
- Each process needs an instance of another resource type, indicated by a request edge.

A state leading to another…

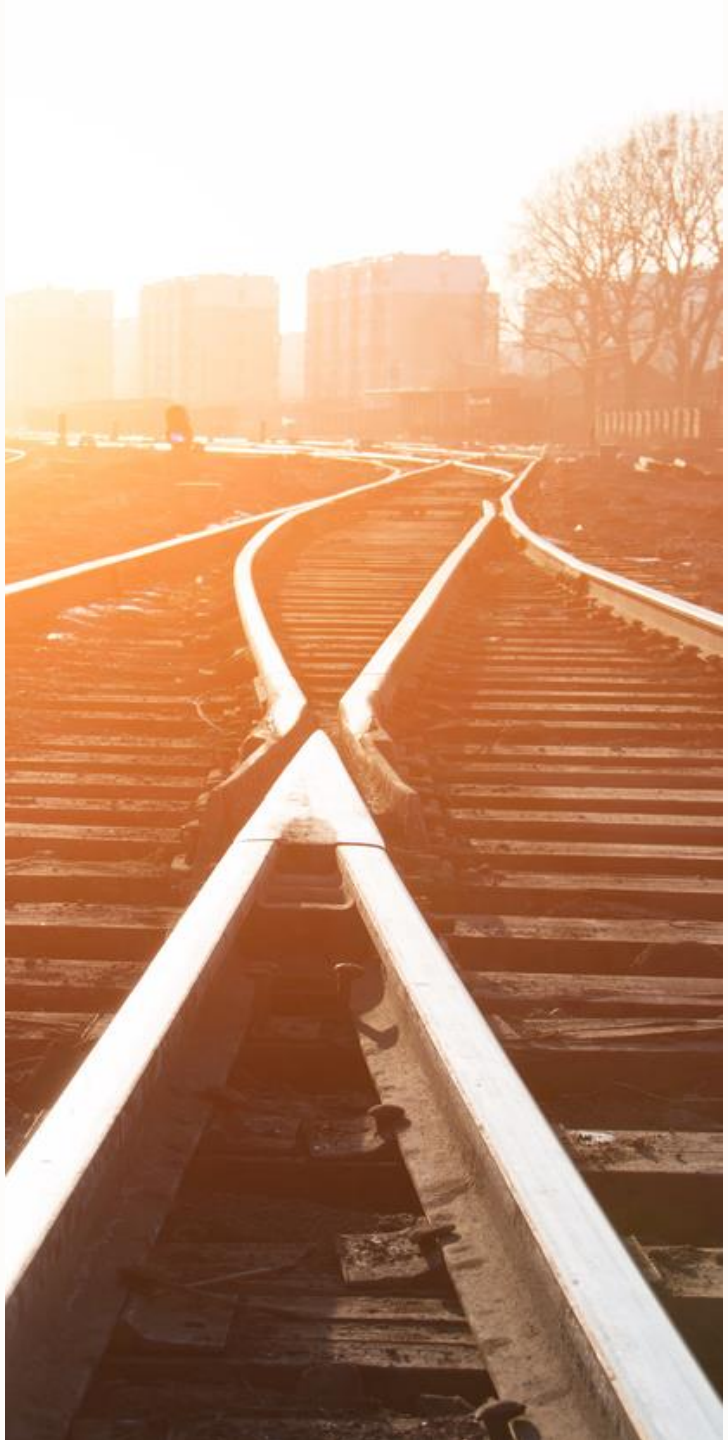


(a) A Unit of R_k Available

(b) All Units of R_k Allocated

Figure 10.19
Representing a State Transition Using the Reusable Resource Graph

- Part (a) is a system state.
- Part (b) is a reusable resource graph for a different state that is reached from the one represented in part (a).
- In Figure 10.19(b), a transition has occurred in which the idle unit of R_k is allocated to P_i.
- Is part (b) a deadlock?



State transitions in a reusable resource graph

- A state transition occurs whenever one of three events occurs:
  1. Any allocated resource is released through the deallocation event, d.
  2. A new resource is requested via the request event, r.
  3. A resource is allocated to a process with the allocate event, a.

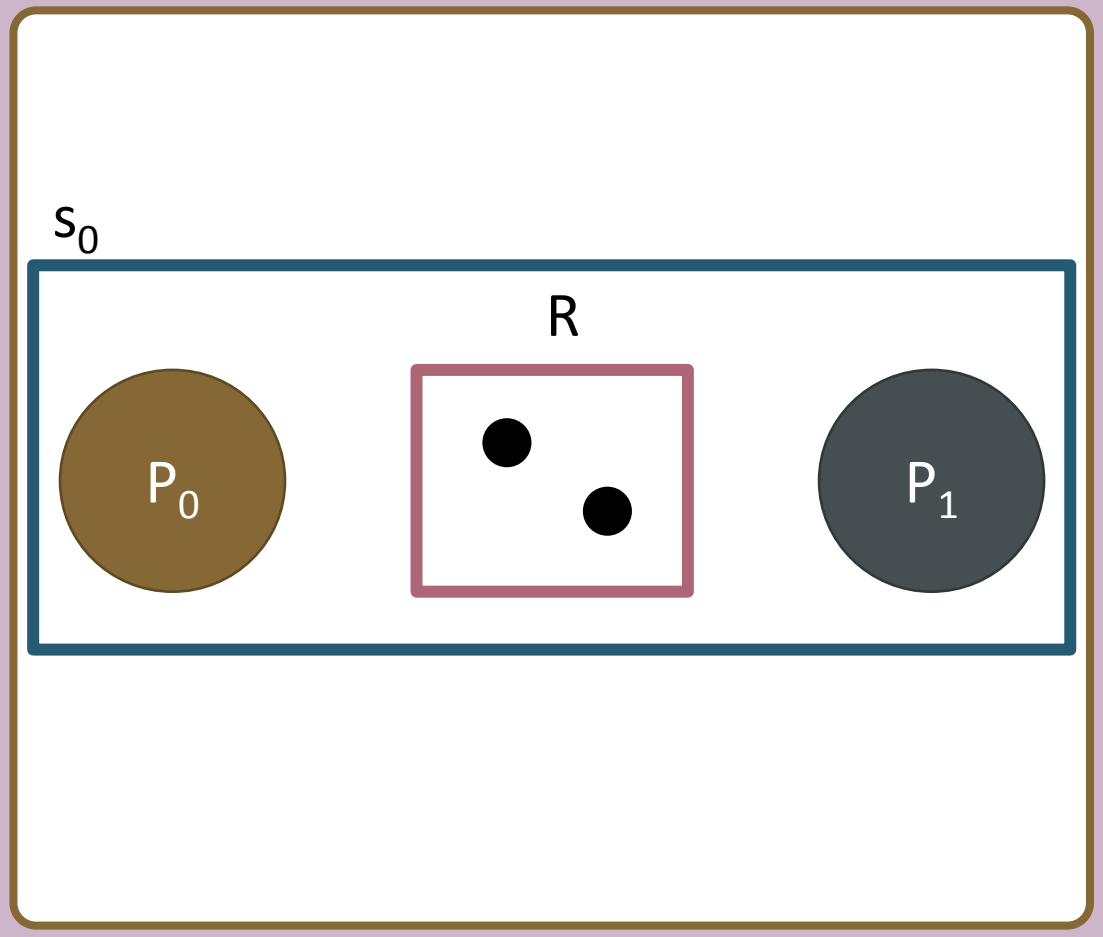
Events that change states

process
is allowed to make a new request only if it is not already waiting for some previous resource request to be granted.

this is called no outstanding request

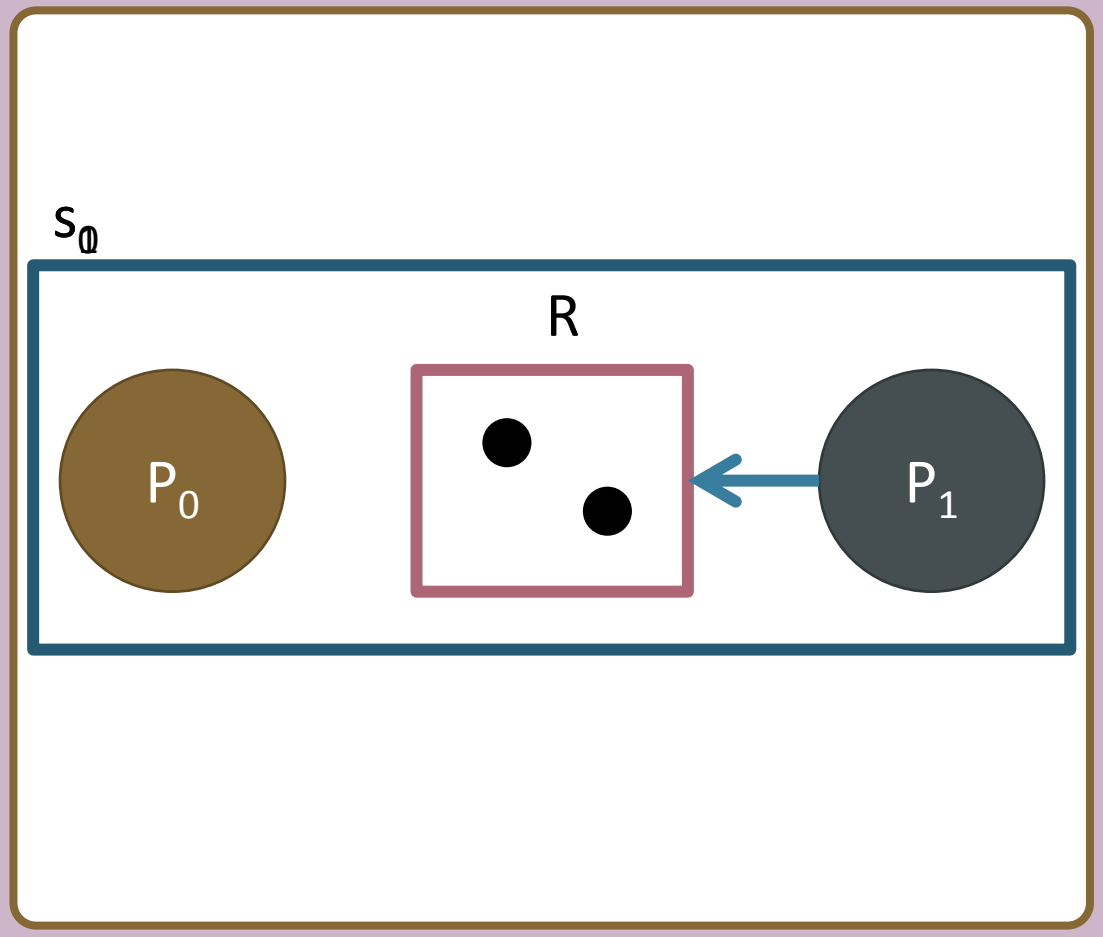
- Request: Assume the system is in state s_i . P_i can request q units of R_j ($q \leq c_j$), provided P_i has no outstanding requests for any resources. A request causes a state transition from s_i to s_j. The reusable resource graph for s_j is created by adding q request edges from P_i to R_j to the graph for s_i.
- Acquisition: Assume the system is in state s_i . P_i can acquire a unit of R_j, if and only if there is a request edge from P_i to R_j. An acquisition causes a state transition from s_i to s_j. The reusable resource graph for s_j is created by changing the request edge from P_i to R_j to an assignment edge from R_j to P_i.
- Release: Assume the system is in state s_i . P_i can release a unit of R_j, if and only if there is an assignment edge from R_j to P_i and there is no request edge from P_i. A release causes a state transition from s_i to s_j. The reusable resource graph for s_j is created by deleting the assignment edge from R_j to P_i.

A reusable resource graph



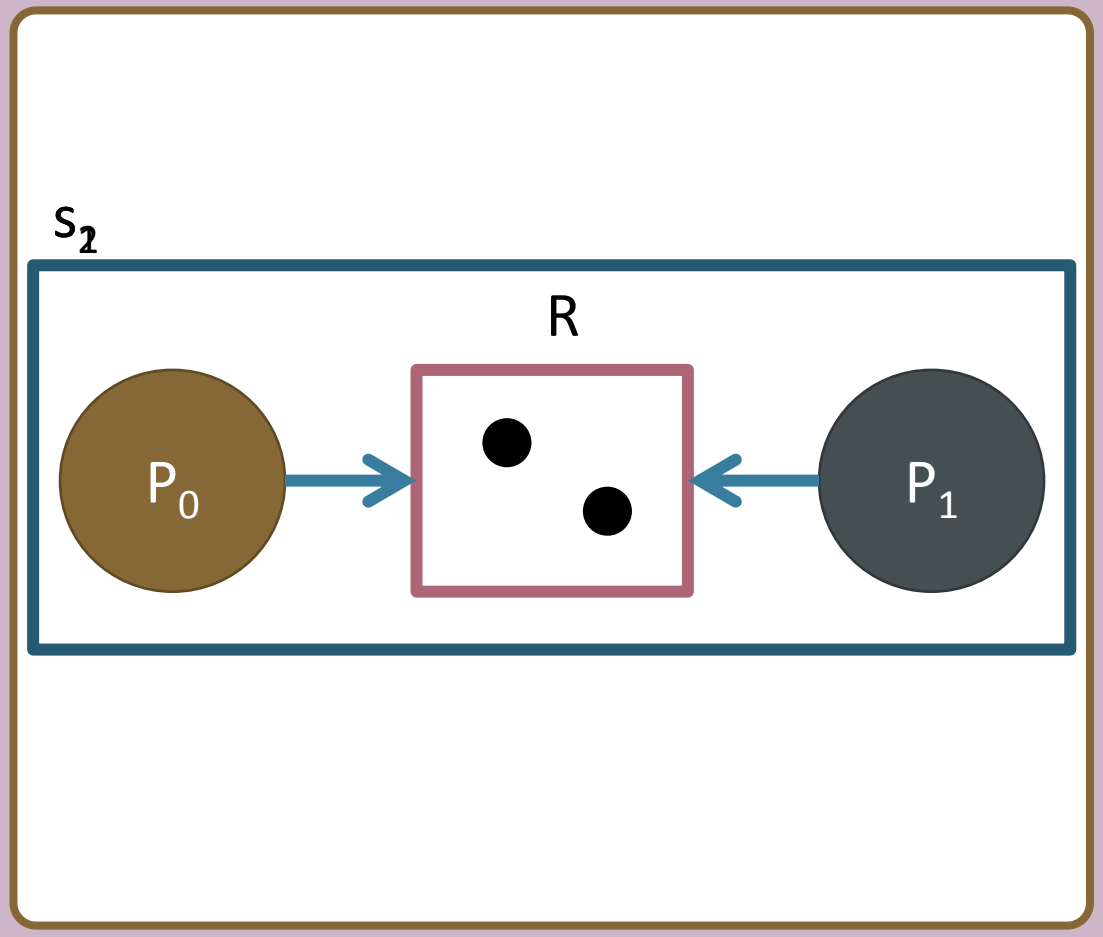
- This is a reusable resource graph.
- There are two processes P_0 and P_1.
- Resource R has two instances.
- The state of the system is s_0.

A reusable resource graph



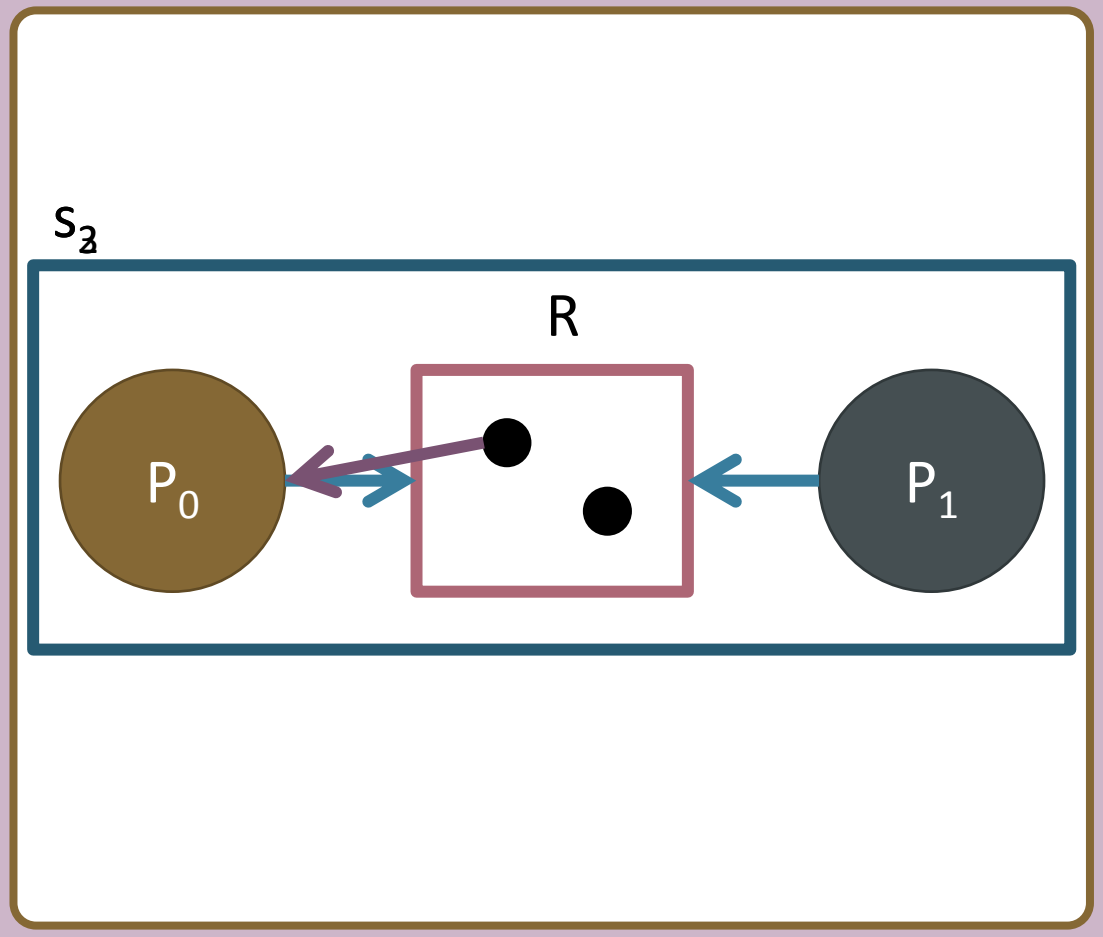
- A request, r_1, is made by P_1 for an instance of R.
- P_1 has no outstanding requests, that is, it is not blocked.
- A request edge is added to s_0.
- The state of the system is s_1.

A reusable resource graph



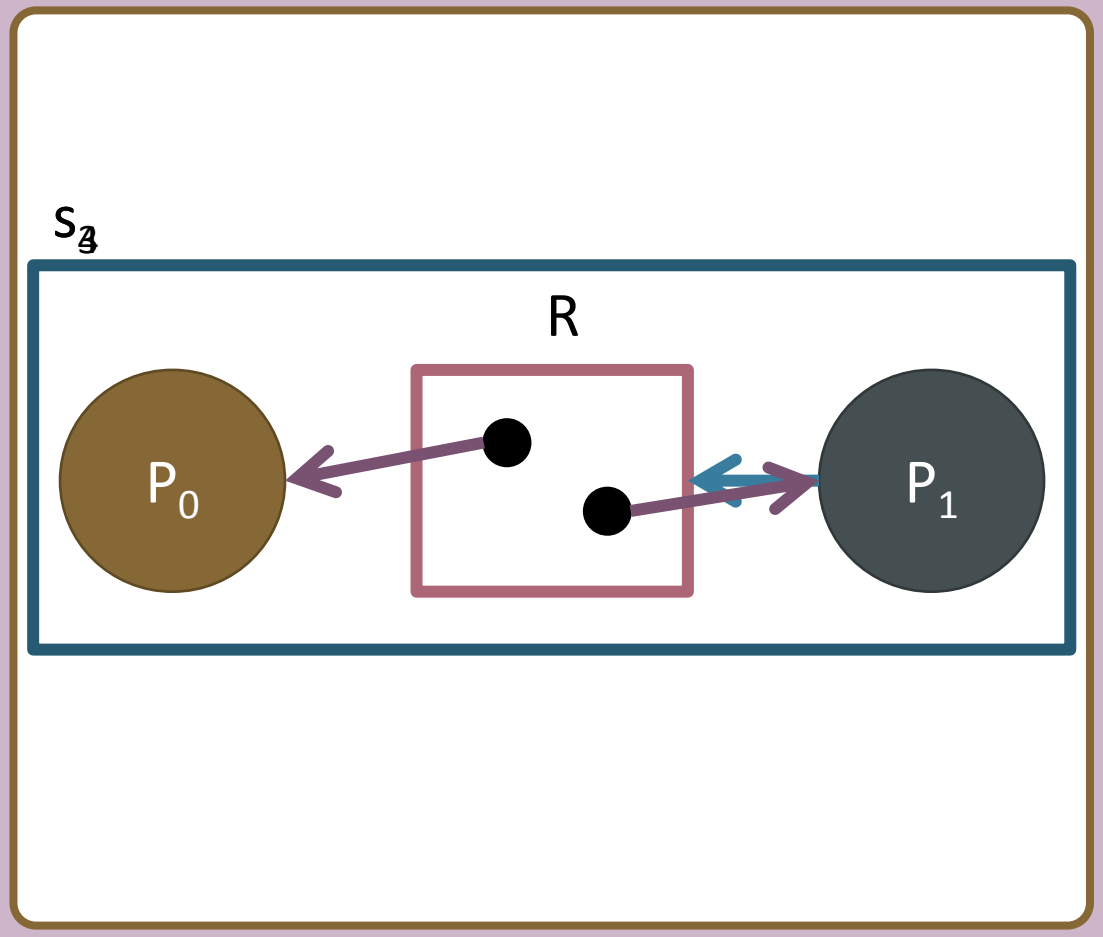
- A request, r_0, is made by P_0 for an instance of R.
- P_0 has no outstanding requests, that is, it is not blocked.
- A request edge is added to s_1.
- The state of the system is s_2.

A reusable resource graph



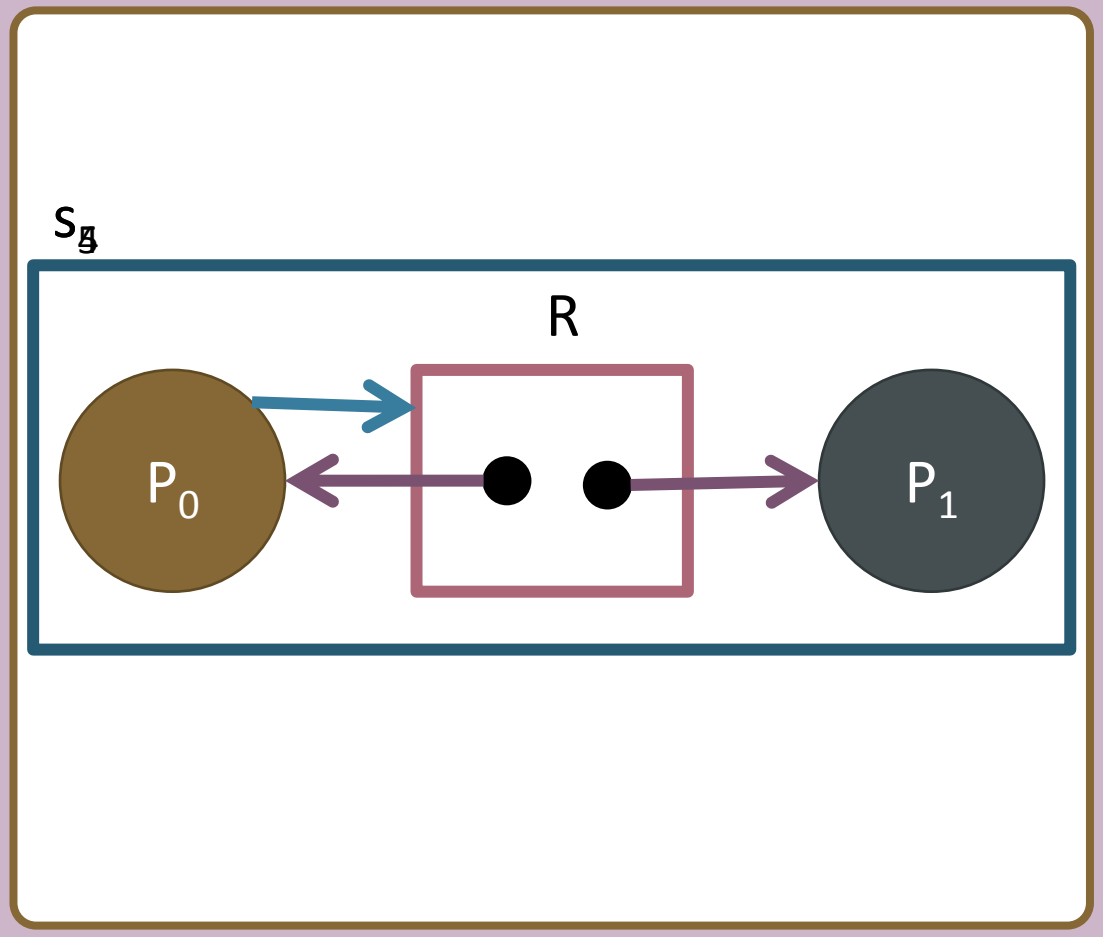
- An allocation, a_0, is made to P_0 of an instance of R.
- The request edge from P_0 to R is removed from s_2.
- An assignment edge is added to s_2.
- The state of the system is s_3.

A reusable resource graph



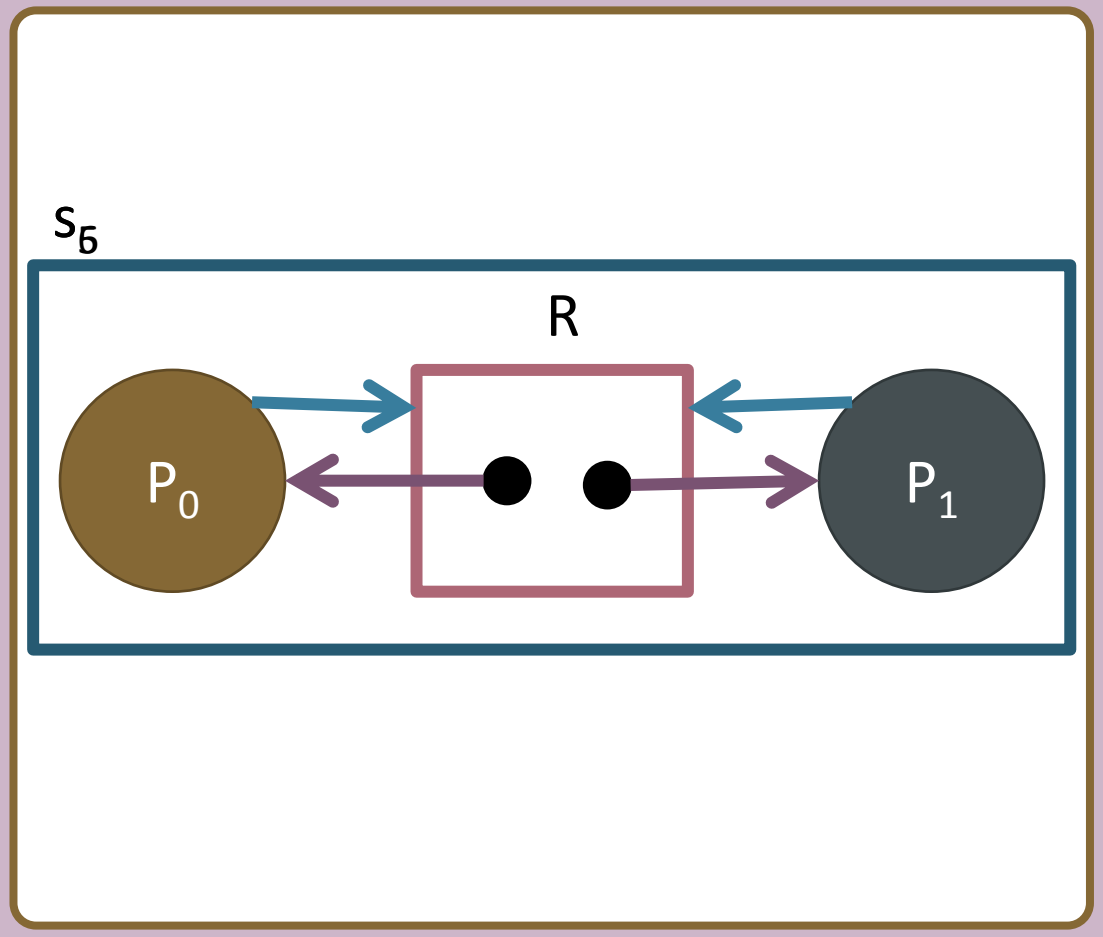
- An allocation, a_1, is made to P_1 of an instance of R.
- The request edge from P_1 to R is removed from s_3.
- An assignment edge is added to s_3.
- The state of the system is s_4.

A reusable resource graph



- A request, r_0, is made by P_0 for an instance of R.
- P_0 has no outstanding requests, that is, it is not blocked.
- A request edge is added to s_4.
- The state of the system is s_5.

A reusable resource graph



- A request, r_1, is made by P_1 for an instance of R.
- P_1 has no outstanding requests, that is, it is not blocked.
- A request edge is added to s_5.
- The state of the system is s_6.
- This is a deadlock!

A set of processes where every process is waiting (directly or indirectly) for another process within the same set, and none of them can proceed.

Cycle ⇒ may or may not be deadlock (depending on the model).
Knot ⇒ deadlock.

Knots in the resource allocation graph

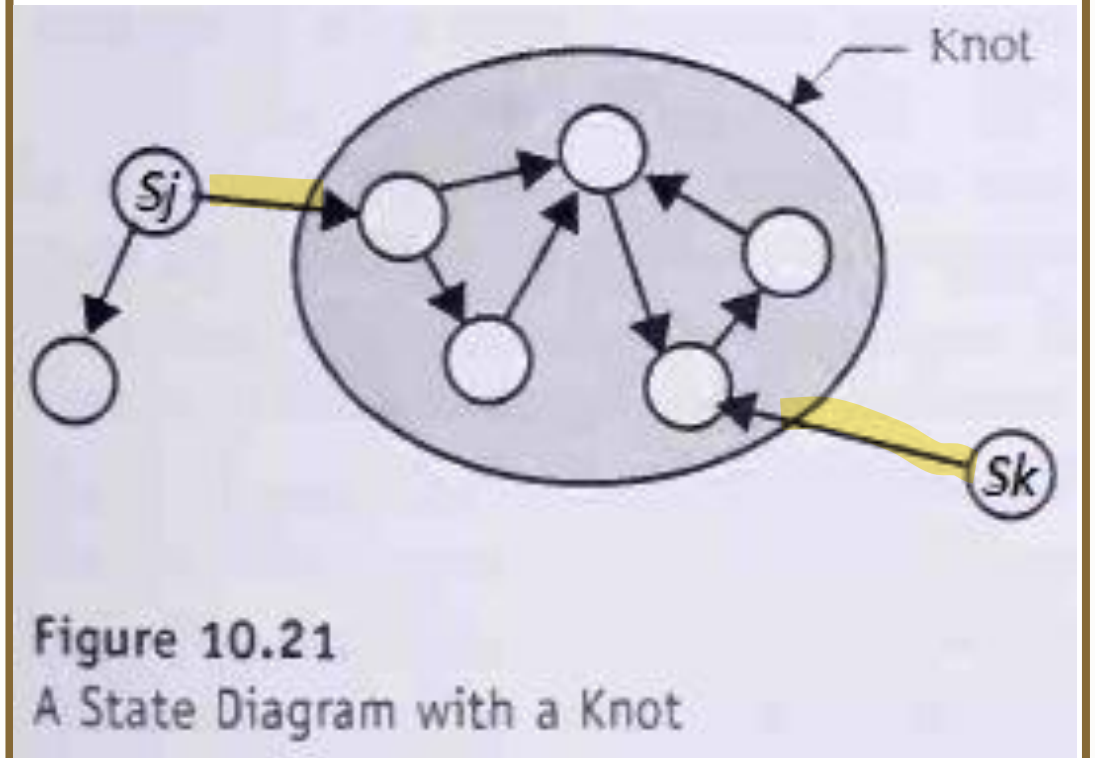


Figure 10.21
A State Diagram with a Knot

- There could be a state diagram that contained a knot and once the system entered any state in the knot, it could never transition to other states outside the knot.
- In the figure shown here, the system can move from state s_j or s_k into the knot, but then it would never be able to change to any of the states outside the knot.
- This situation complicates the algorithm that checks the resource allocation graph to see if a process is deadlocked.

How to detect a deadlock?

- To detect if s_k is a deadlock state, we must be assured that there is no sequence of transitions unblocking all blocked processes.
- We can consider all transformations of the reusable resource graph to determine if there is a new graph reachable from it.
- If we can find a series of transformations in which a process P_i is unblocked, the original state is not a deadlock.

Graph reduction

- A graph reduction is a set of transformations representing optimal action by processes.
- The intent is to decide whether the current state is a deadlock.
- A serially reusable resource graph can be reduced by P_i if these conditions are met:
  - P_i is not blocked.
  - P_i has no request edges.
  - There are assignment edges directed into P_i
- The reduction transforms the reusable resource graph by removing all assignment edges to P_i.

Reducibility of a resource allocation graph

- A reusable resource graph is irreducible if it cannot be reduced by any process.
- A reusable resource graph is completely reducible if there is a sequence of reductions that leads to a graph having no edges of any kind.
- It can be proven that given a reusable resource graph representing state s_k, state s_k is a deadlock state, if and only if, the serially reusable resource graph is not completely reducible.

Cycle and graph reduction

if r1 given to p0, it can resolve, after that p1 can work

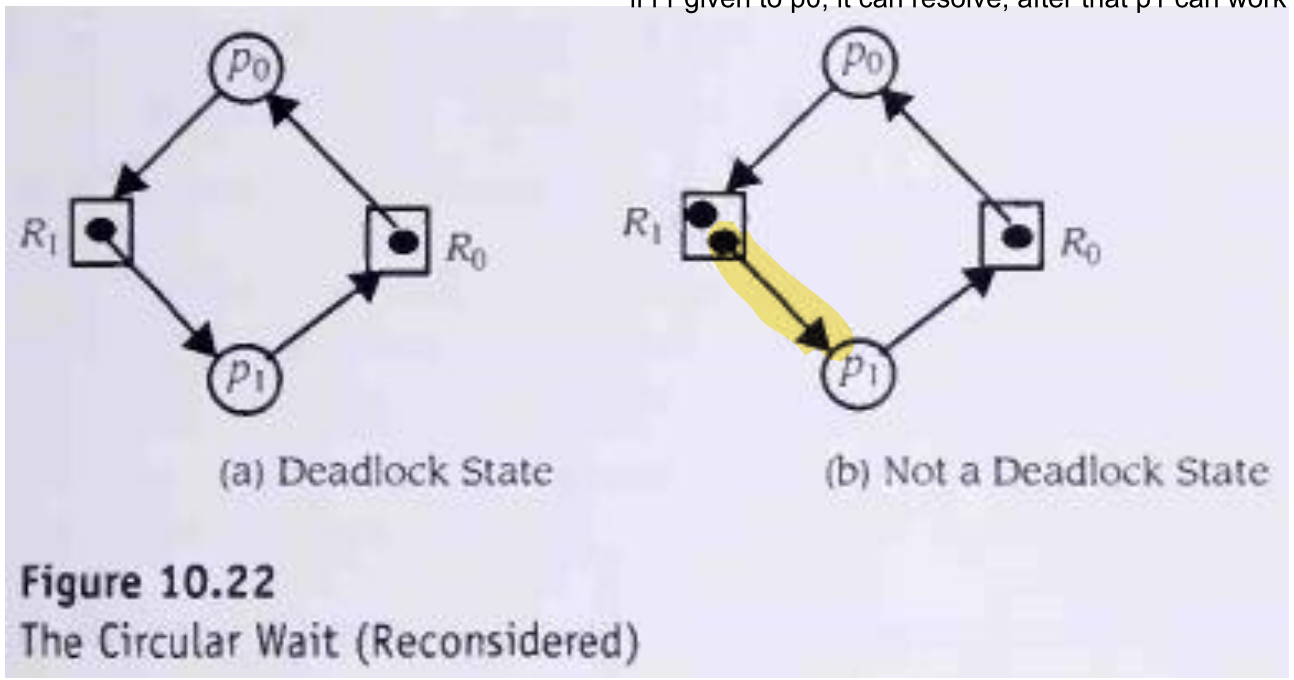
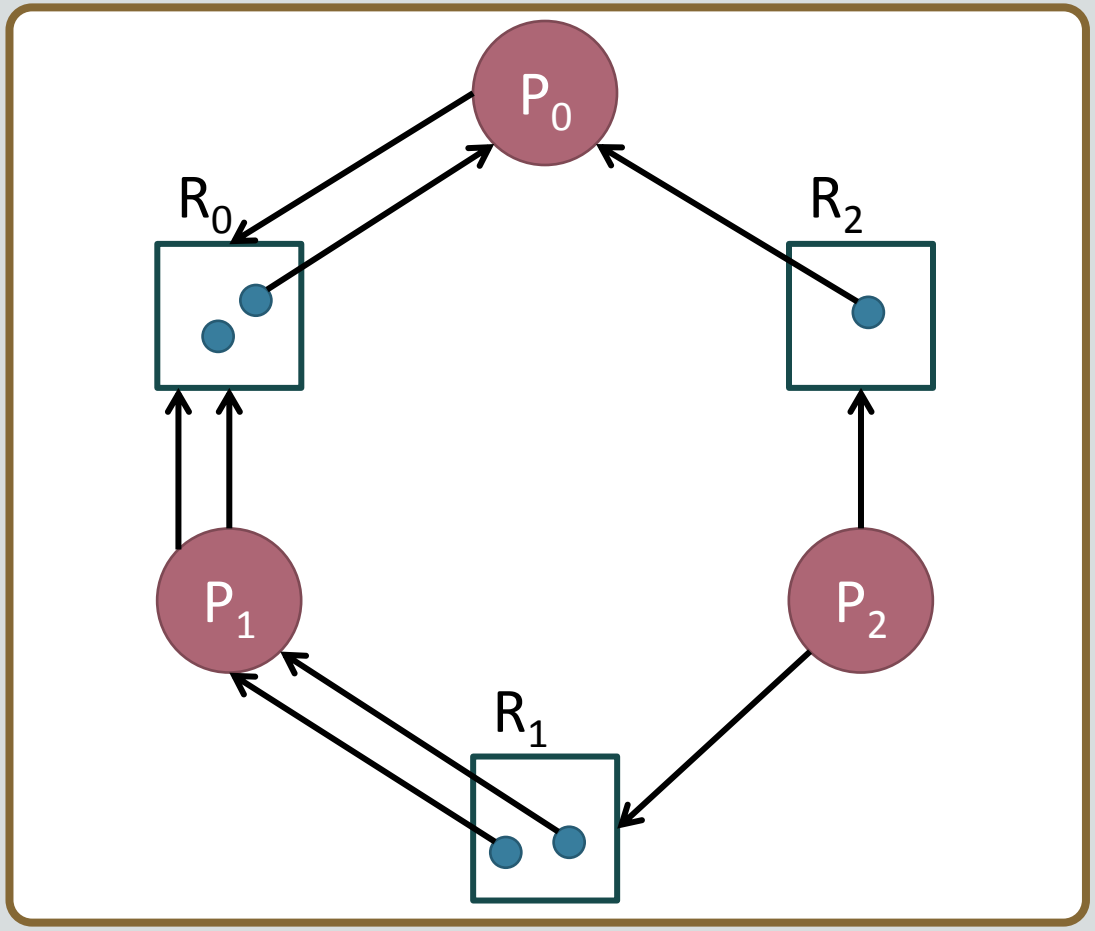


Figure 10.22
The Circular Wait (Reconsidered)

- The graph of a deadlock state must contain a cycle.
- A cycle in the reusable graph is a necessary condition for deadlock. However, it is not a sufficient condition for deadlock.
- There is a deadlock in part (a) of Figure 10.22. But part (b) is not a deadlock state. Yet it too contains the same cycle.

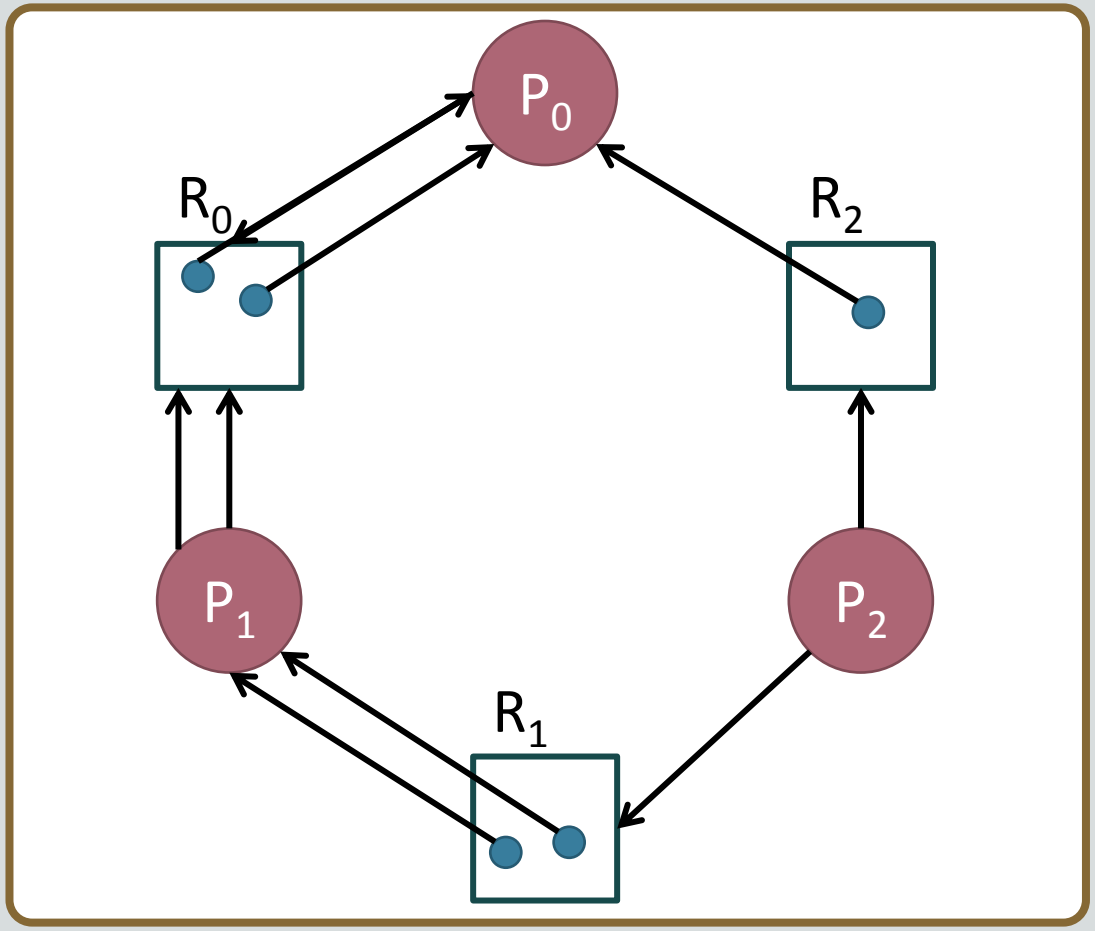
request cannot be granted, process blocked

p0 not waiting for any assigned resource, if r0 goven to p0, it will resolve after some time



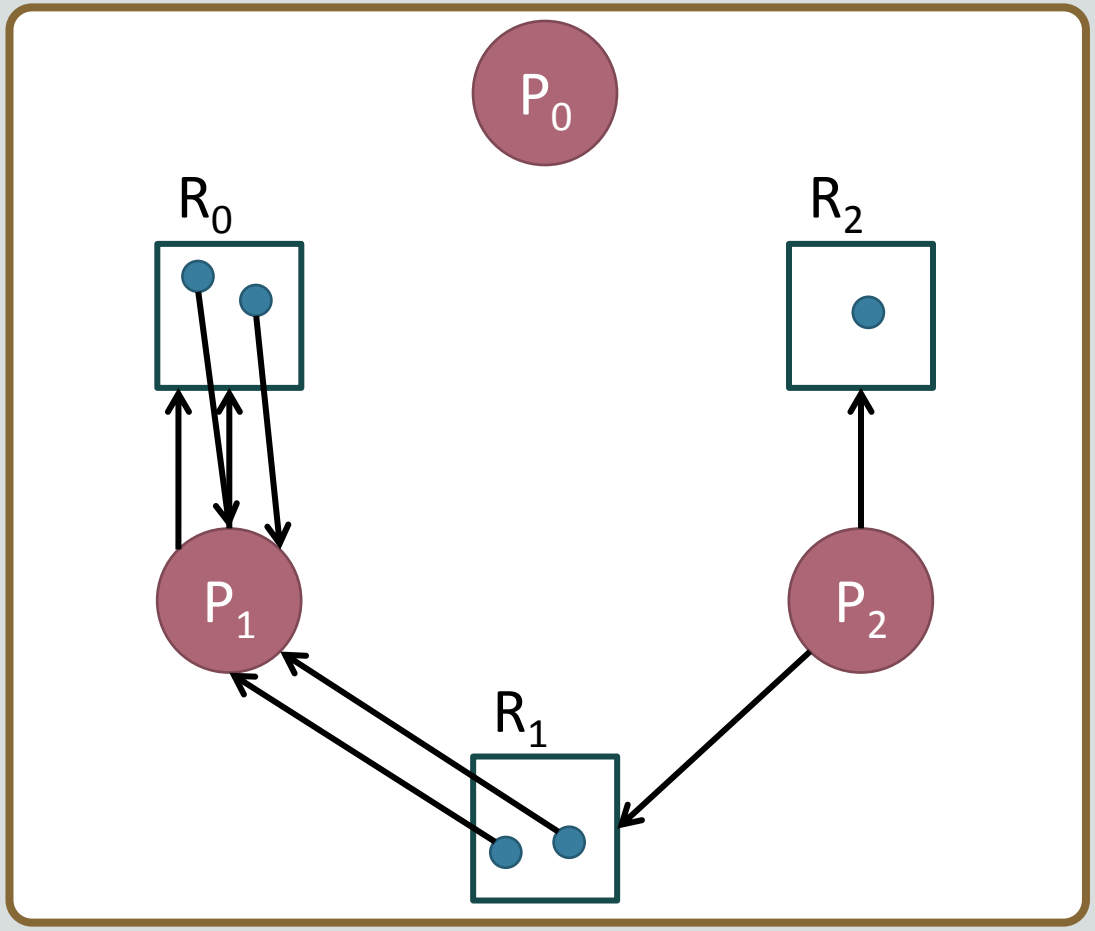
Is this reusable resource graph deadlocked?

- To answer this question, we attempt to reduce the given graph.
- Identify the blocked processes.
  - P_1 has two requests, one of which cannot be granted, so it is blocked.
  - P_2 has a request which cannot be granted, so it is blocked.
- P_0 has a request which can be granted!



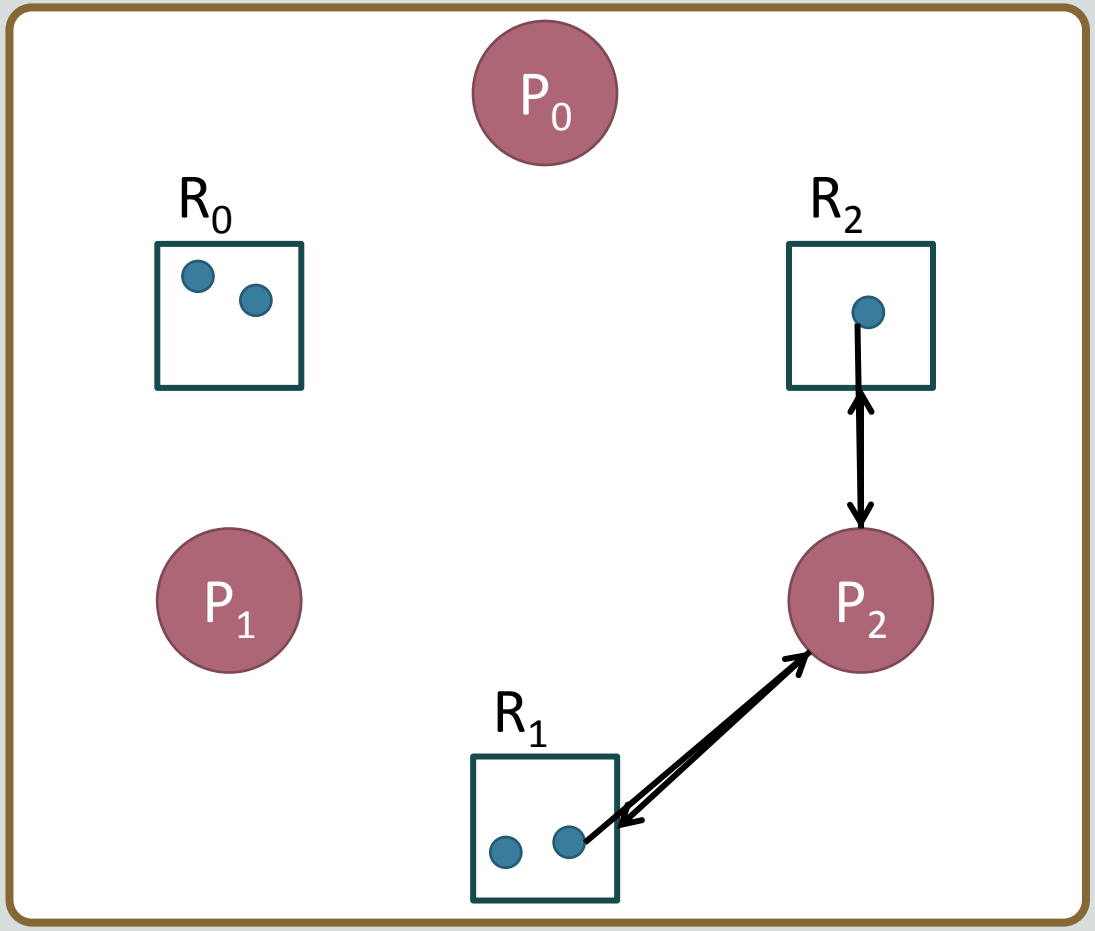
Is this reusable resource graph deadlocked?

- Reducing the graph by P_0:
  - Convert the request edge to an assignment edge.
  - P_0 has assignment edges only.
  - P_0 releases all its allocated resources.
- P_1 has unblocked!



Is this reusable resource graph deadlocked?

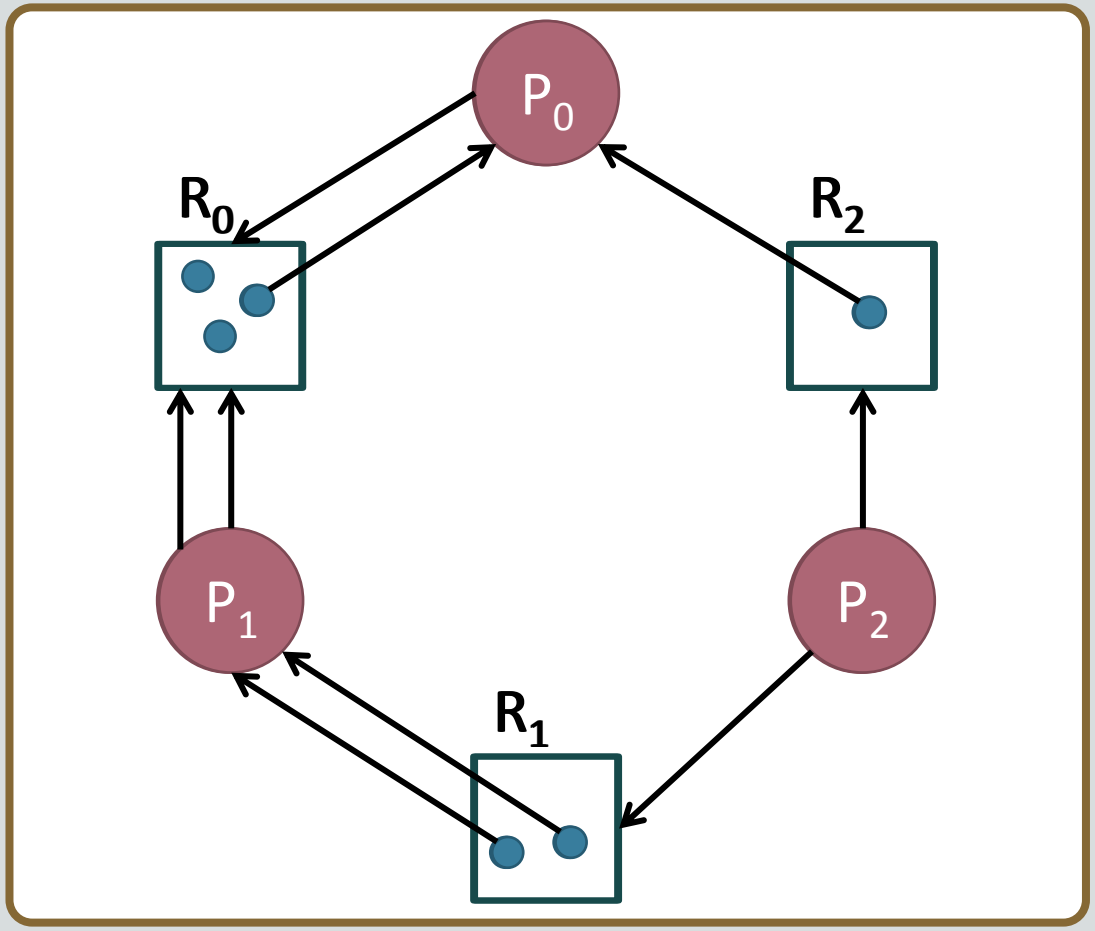
- Reducing the graph by P_1:
  - Convert the request edges to assignment edges.
  - P_1 has assignment edges only.
  - P_1 releases all its allocated resources.
- P_2 has unblocked!



Is this reusable resource graph deadlocked?

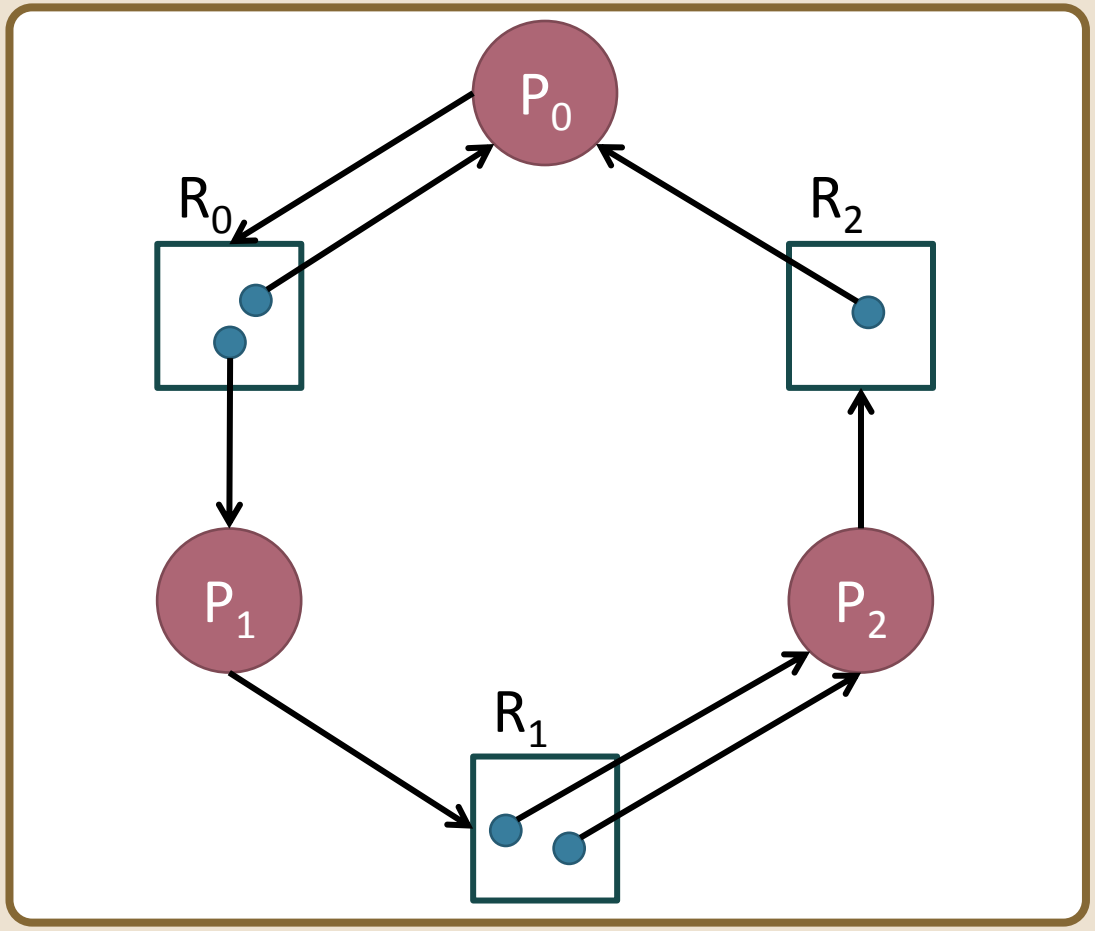
- Reducing the graph by P_2:
  - Convert the request edge to an assignment edge.
  - P_2 has assignment edges only.
  - P_2 releases all its allocated resources.
- The graph has NO edges.

Reduction sequence: <P_0, P_1, P_2>



Is this reusable resource graph deadlocked?

- We say that the graph has been completely reduced.
- This shows that the initial serially reusable resource graph did not contain a deadlock.

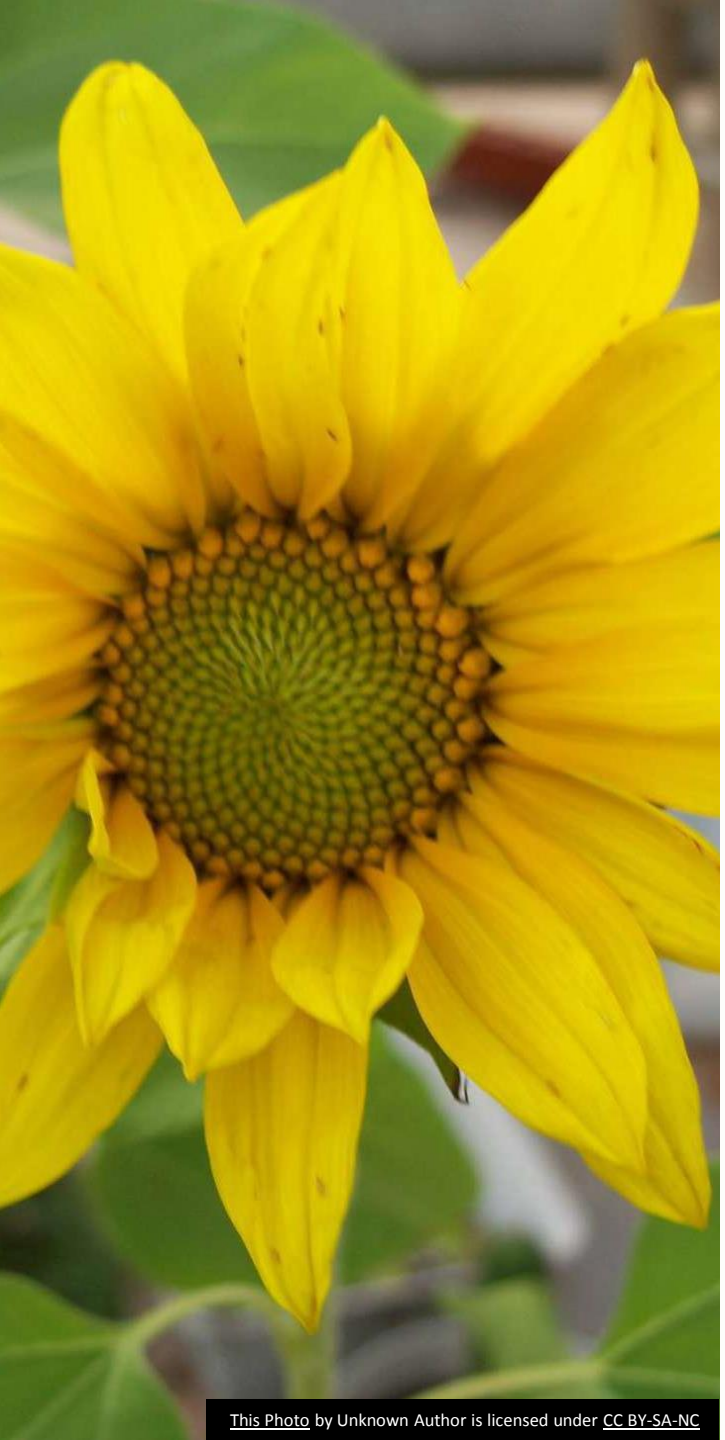


Is this reusable resource graph deadlocked?

- To answer this question, we attempt to reduce the given graph.
- Identify the blocked processes.
  - P_0 has a request which cannot be granted, so it is blocked. The same is true for P_1 and P_2.
- The graph is not reducible by any process.
- The given state is a deadlock.

Consumable resources

- Consumable resources differ from serially reusable resources in that a process may request consumable resources but will never release them. Conversely, a process can release units of a consumable resource without ever acquiring them.
- Because such resources may have an unbounded number of units and because allocated units are not released, the model for analyzing serially reusable resources does not apply to consumable resources.
- However, by our redefining the model for consumable resources, conditions can be found to test a system state for deadlock.



Units of a consumable resource

- A consumable resource, R_j, has an unbounded number of identical units such that the following holds:
  - The number of units of the resource, w_j varies.
  - There are one or more producer processes, P_p, that may increase w_j by releasing units of the resource.
  - Consumer processes, P_c, decrease w_j by acquiring units of the resource.

produer produces the units, consumers consumes it

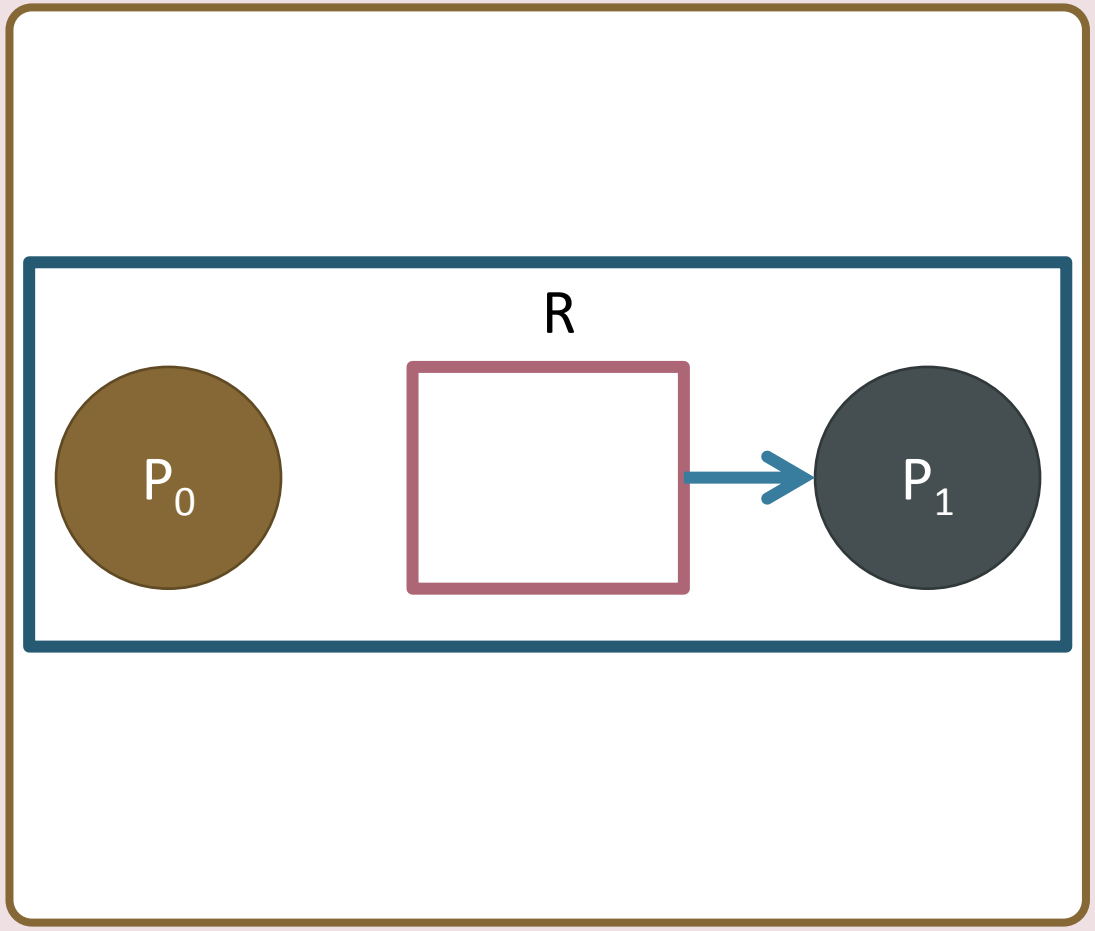
Consumable resource graph models

- A consumable resource graph is a directed graph such that the following is true:
- The n + m nodes represent n processes and m resources.
- An edge from P_i to R_j is a request edge, which represents a request for 1 unit of R_j by P_i.
- An edge from R_j to P_i is a producer edge, which identifies P_i as the producer of R_j.
- The number of units of R_j is w_j that is graphically represented by small tokens inside R_j.

Events that change states

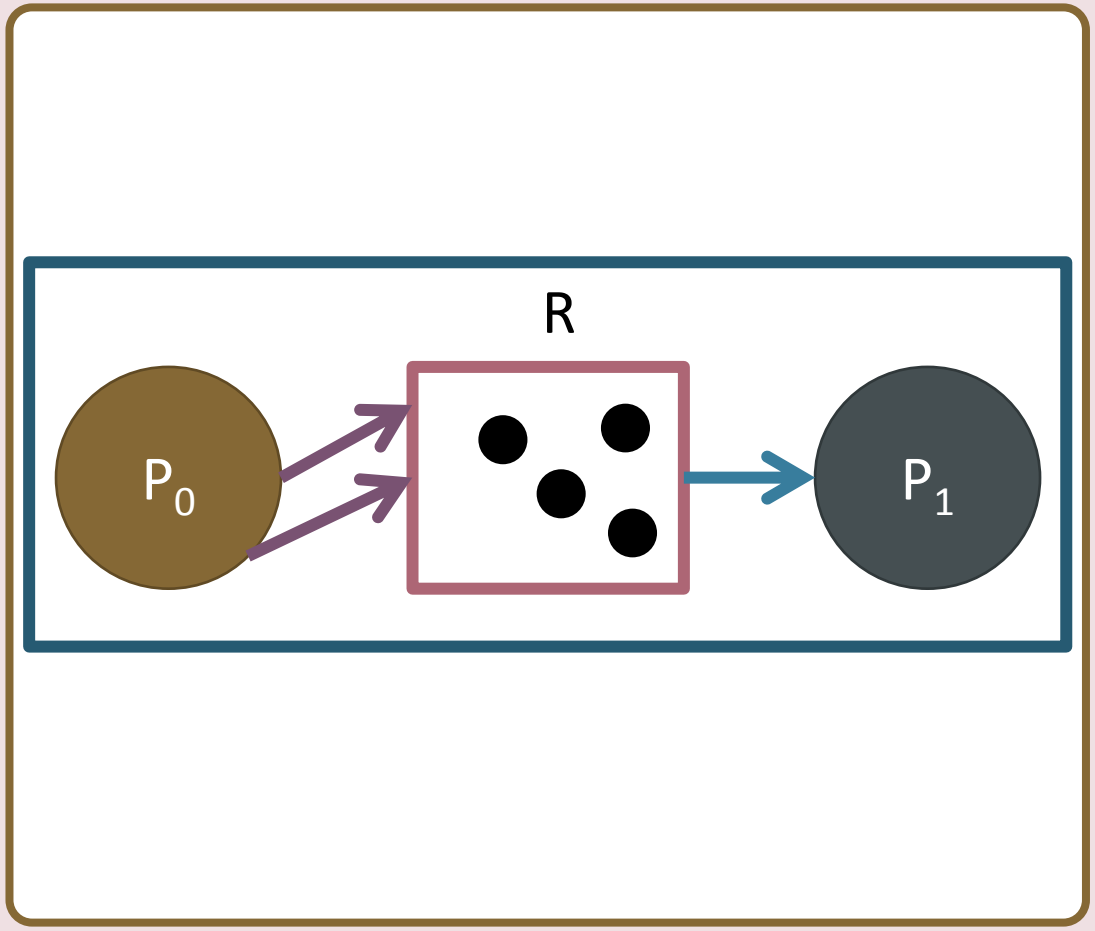
- Request: Assume the system is in state s_i . P_i can request any number of units, q, of R_j, provided P_i has no outstanding requests for any resources. A request causes a state transition from s_i to s_j. The consumable resource graph for s_j is created by adding q request edges from P_i to R_j to the graph for s_i.
- Acquisition: Assume the system is in state s_i . P_i can acquire a unit of R_j, if and only if there is a request edge from P_i to R_j. An acquisition causes a state transition from s_i to s_j. The consumable resource graph for s_j is created by deleting the request edge from P_i to R_j and by decrementing w_j.
- Release: Assume the system is in state s_i . P_i can release units of R_j, if and only if there is a producer edge from R_j to P_i and there is no request edge from P_i. A release causes a state transition from s_i to s_j. The consumable resource graph for s_j is created by incrementing w_j.

A consumable resource graph



- This is a consumable resource graph.
- There are two processes P_0 and P_1.
- P_1 is the producer of R.
- Resource R has no instances, as it has not been produced yet.

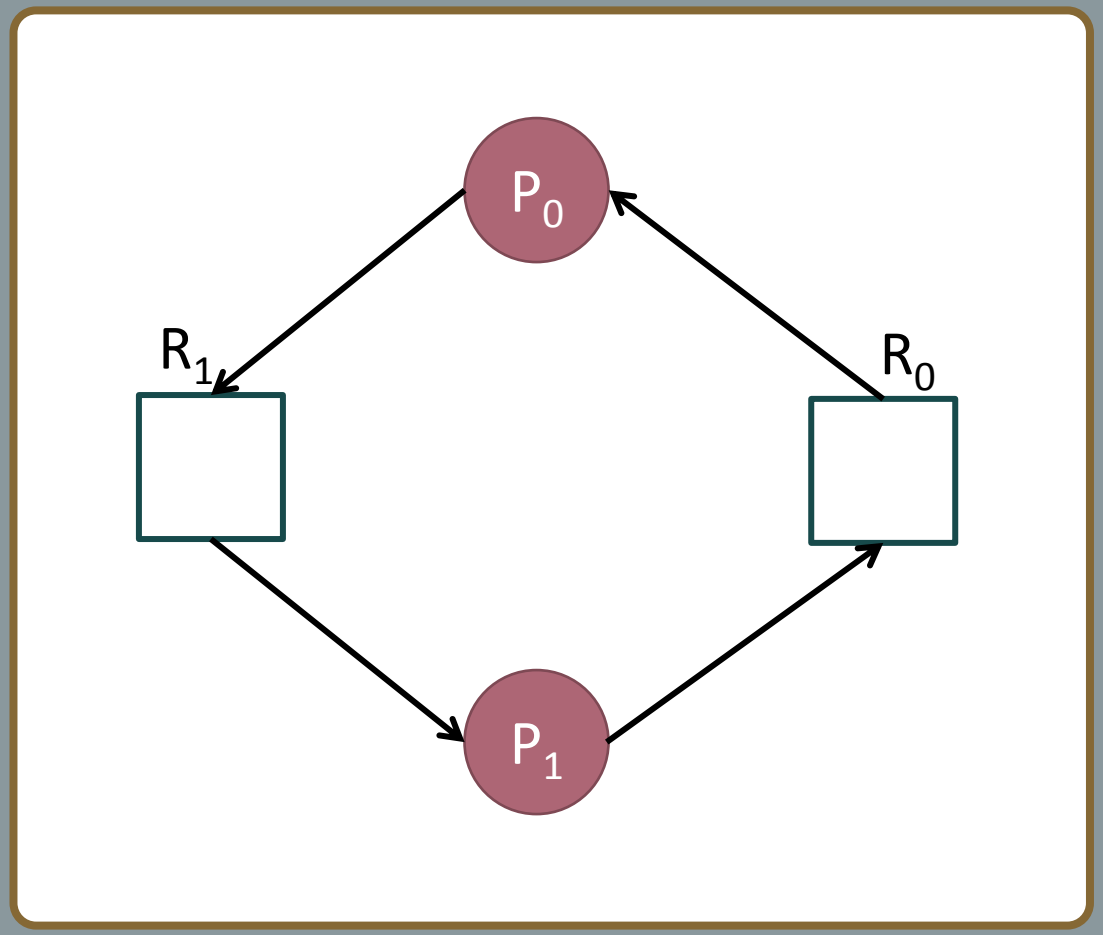
A consumable resource graph



- P_1 is not blocked.
- P_1 produces an instance of R.
- P_0 requests 2 instances of R.
- P_0 is blocked now.
- P_1 producers 3 instances of R.
- P_0 is granted its requests.
- The system is not deadlocked.

Graph reduction

- A graph reduction is a set of transformations representing optimal action by processes.
- The intent is to decide whether the current state is a deadlock.
- A consumable resource graph can be reduced by P_i if P_i is not blocked.
- The reduction transforms the consumable resource graph by removing request edges of P_i, and decrementing w_j once for each request edge from P_i to R_j.
- If there is a producer edge from a resource R_j to P_i the reduction releases an unbounded number of units of R_j and deletes the producer edge, $R_j \rightarrow P_i$, from the graph. Instead of small tokens, we show the symbol of infinity ∞ inside the node R_j.

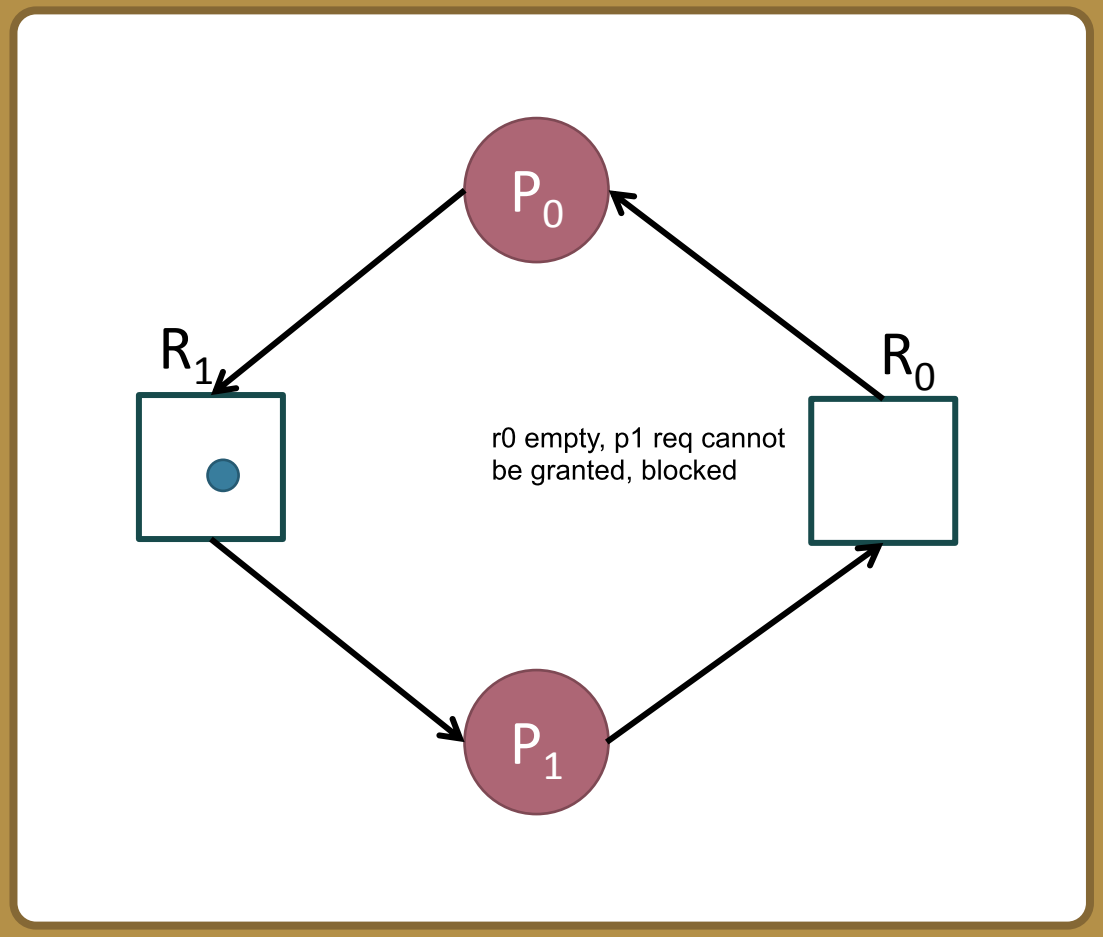


Is this consumable resource graph deadlocked?

- To answer this question, we attempt to reduce the given graph.
- Identify the blocked processes.
  - P_0 has a request which cannot be granted, so it is blocked. So is P_1.
- The graph is not reducible by any process.
- The given state is a deadlock.

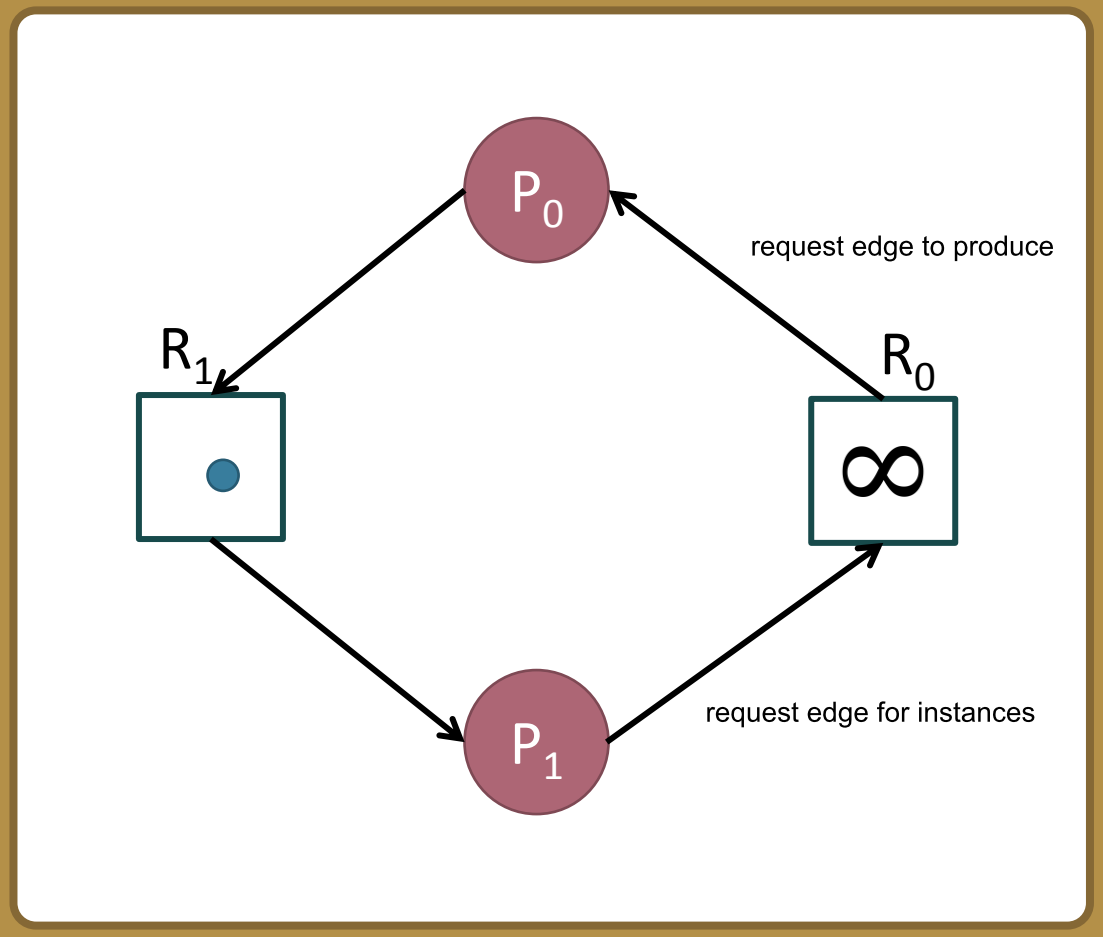
empty resorces

Is this consumable resource graph deadlocked?



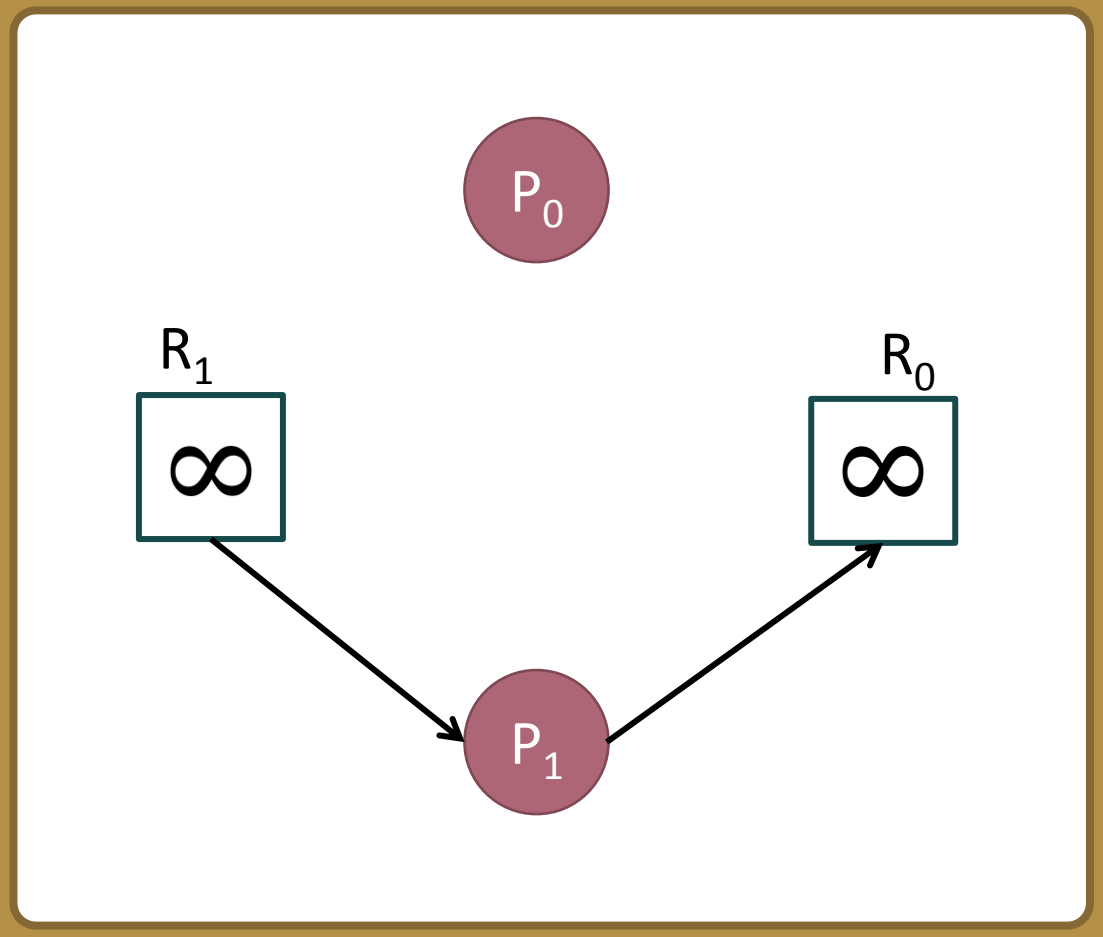
- To answer this question, we attempt to reduce the given graph.
- Identify the blocked processes.
  - P_1 has a request which cannot be granted, so it is blocked.
- P_0 has a request which can be granted!

Is this consumable resource graph deadlocked?



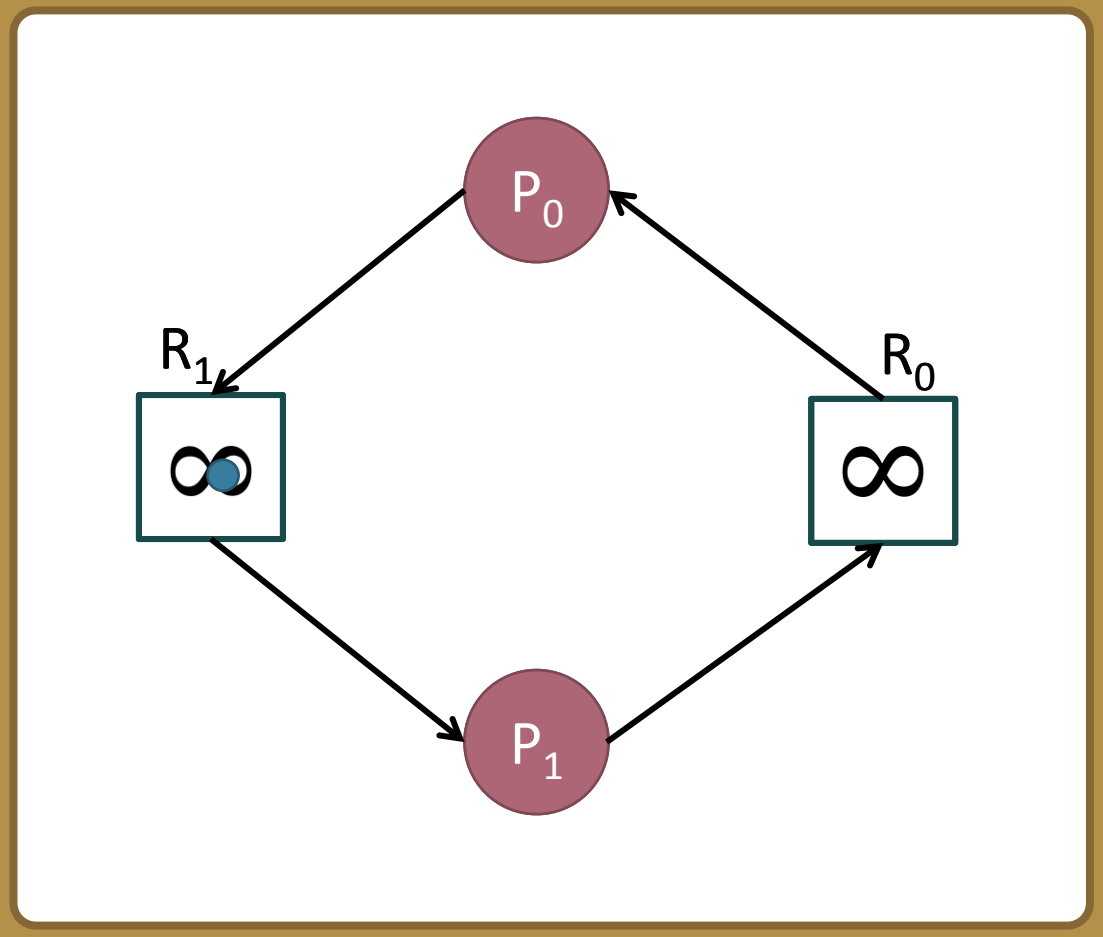
- Reducing the graph by P_0:
  - Remove the request edge, decrementing an instance of R_1.
  - Being the producer of R_0, P_0 produces infinite instances of R_0. The producer edge is removed.
- P_1 has unblocked!

Is this consumable resource graph deadlocked?



- Reducing the graph by P_1:
  - Remove the request edge, decrementing an instance of R_0.
  - Being the producer of R_1, P_1 produces infinite instances of R_1. The producer edge is removed.
- The graph has NO edges.

Reduction sequence: <P_0, P_1>



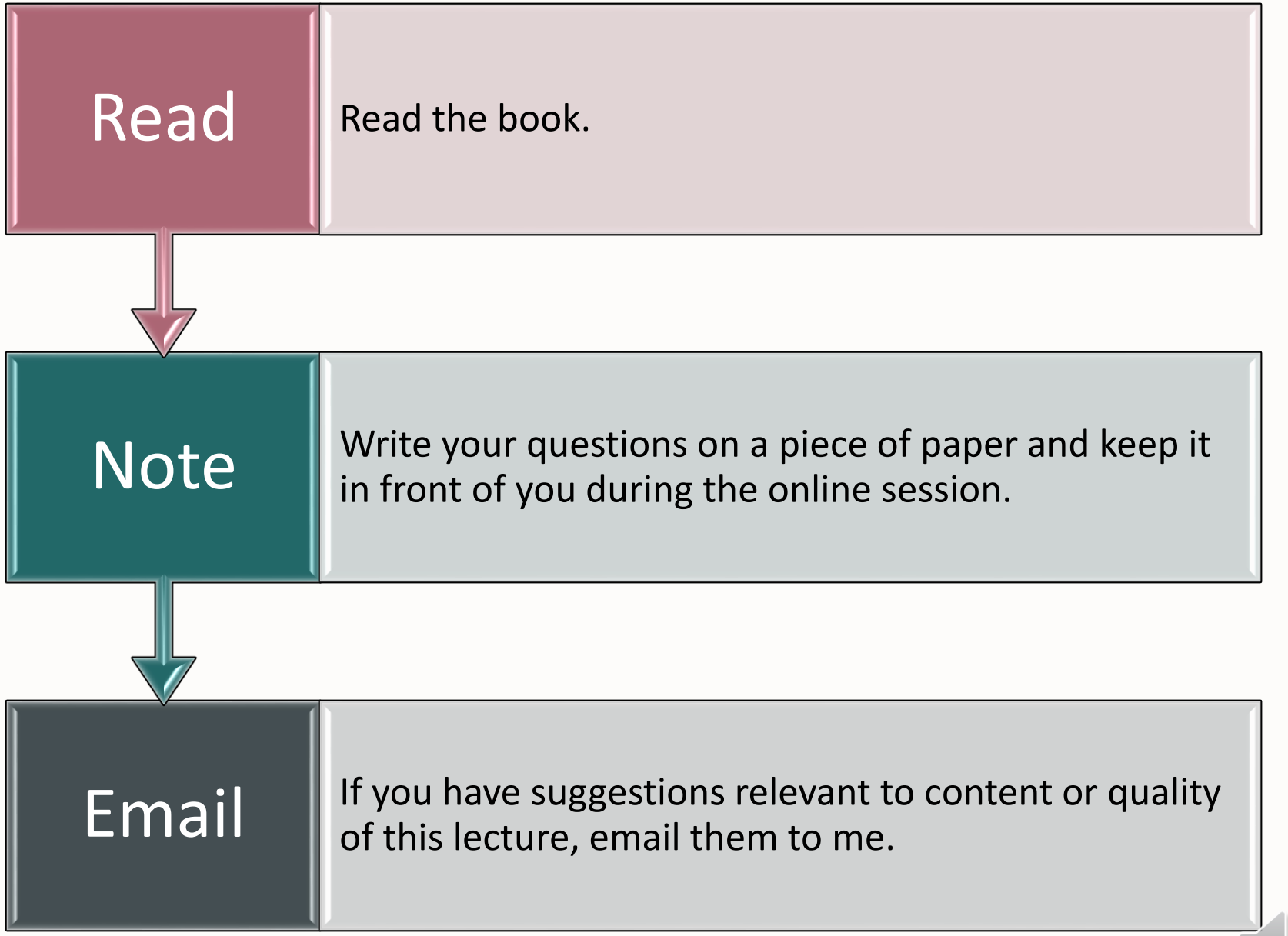
Is this consumable resource graph deadlocked?

- We say that the graph has been completely reduced.
- This shows that the initial consumable resource graph did not contain a deadlock.

Lessons from the lecture

- We covered more examples of the banker’s algorithm.
- We realized that deadlock avoidance was still more expensive than detection and recovery.
- We explored reusable resource graph models.
- We learned how to detect deadlock for reusable resources.
- We started the discussion on consumable resource graph models.

Things to do


Read
Read the book.
Note
Write your questions on a piece of paper and keep it in front of you during the online session.
Email
If you have suggestions relevant to content or quality of this lecture, email them to me.